

10. Address Translation Services (ATS) §

10.1 ATS Architectural Overview §

Most contemporary system architectures make provisions for translating addresses from DMA (bus mastering) I/O Functions. In many implementations, it has been common practice to assume that the physical address space seen by the CPU and by an I/O Function is equivalent. While in others, this is not the case. The address programmed into an I/O Function is a “handle” that is processed by the Root Complex (RC). The result of this processing is often a translation to a physical memory address within the central complex. Typically, the processing includes access rights checking to insure that the DMA Function is allowed to access the referenced memory location(s).

The purposes for having DMA address translation vary and include:

- Limiting the destructiveness of a “broken” or misprogrammed DMA I/O Function
- Providing for scatter/gather
- Ability to redirect message-signaled interrupts (e.g., MSI or MSI-X) to different address ranges without requiring coordination with the underlying I/O Function
- Address space conversion (32-bit I/O Function to larger system address space)
- Virtualization support

Irrespective of the motivation, the presence of DMA address translation in the host system has certain performance implications for DMA accesses.

Depending on the implementation, DMA access time can be significantly lengthened due to the time required to resolve the actual physical address. If an implementation requires access to a main-memory-resident translation table, the access time can be significantly longer than the time for an untranslated access. Additionally, if each transaction requires multiple memory accesses (e.g., for a table walk), then the memory transaction rate (i.e., overhead) associated with DMA can be high.

To mitigate these impacts, designs often include address translation caches in the entity that performs the address translation. In a CPU, the address translation cache is most commonly referred to as a translation look-aside buffer (TLB). For an I/O TA, the term address translation cache or ATC is used to differentiate it from the translation cache used by the CPU.

While there are some similarities between TLB and ATC, there are important differences. A TLB serves the needs of a CPU that is nominally running one thread at a time. The ATC, however, is generally processing requests from multiple I/O Functions, each of which can be considered a separate thread. This difference makes sizing an ATC difficult depending upon cost models and expected technology reuse across a wide range of system configurations.

The mechanisms described in this specification allow an I/O Device to participate in the translation process and provide an ATC for its own memory accesses. The benefits of having an ATC within a Device include:

- Ability to alleviate TA resource pressure by distributing address translation caching responsibility (reduced probability of “thrashing” within the TA)
- Enable ATC Devices to have less performance dependency on a system’s ATC size
- Potential to ensure optimal access latency by sending pretranslated requests to central complex

This specification will provide the interoperability that allows PCIe Devices to be used in conjunction with a TA, but the TA and its Address Translation and Protection Table (ATPT) are treated as implementation specific and are outside the

scope of this specification. While it may be possible to implement ATS within other PCIe Components, this specification is confined to PCIe Devices and PCIe Root Complex Integrated Endpoints (RCiEPs).

§ Figure 10-1 illustrates an example platform with a TA and ATPT, along with a set of PCIe Devices and RC Integrated Endpoints with integrated ATC. A TA and an ATPT are implementation specific and can be distinct or integrated components within a given system design.

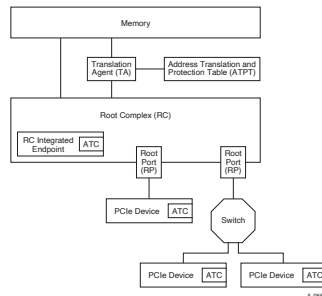


Figure 10-1 Example Illustrating a Platform with TA, ATPT, and ATC Elements §

10.1.1 Address Translation Services (ATS) Overview §

The ATS chapter provides a new set of TLP and associated semantics. ATS uses a request-completion protocol between a Device²⁰⁸ and a Root Complex (RC) to provide translation services. In addition, a new AT field is defined within the Memory Read and Memory Write TLP. The new AT field enables an RC to determine whether a given request has been translated or not via the ATS protocol.

§ Figure 10-2 illustrates the basic flow of an ATS Translation Request operation.

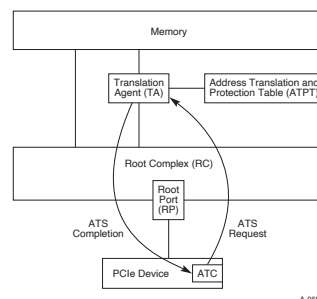


Figure 10-2 Example ATS Translation Request/Completion Exchange §

In this example, a Function-specific work request is received by a single-Function PCIe Device. The Function determines through an implementation specific method that caching a translation within its ATC would be beneficial. There are a number of considerations a Function or software can use in making such a determination; for example:

- Memory address ranges that will be frequently accessed over an extended period of time or whose associated buffer content is subject to a significant update rate

208. All references within this chapter to a Device apply equally to a PCIe Device or a Root Complex Integrated Endpoint. ATS does not delineate between these two types in terms of requirements, semantics, configuration, error handling, etc. From a software perspective, an ATS-capable Root Complex Integrated Endpoint must behave the same as an ATS-capable non-integrated Device.

- Memory address ranges, such as work and completion queue structures, data buffers for low-latency communications, graphics frame buffers, host memory that is used to cache Function-specific content, and so forth

Given the variability in designs and access patterns, there is no single criteria that can be applied.

The Function generates an ATS Translation Request which is sent upstream through the PCIe hierarchy to the RC which then forwards it to the TA. An ATS Translation Request uses the same routing and ordering rules as defined in this specification. Further, multiple ATS Translation Requests can be outstanding at any given time; i.e., one may pipeline multiple requests on one or more TC. Each TC represents a unique ordering domain and defines the domain that must be used by the associated ATS Translation Completion.

Upon receipt of an ATS Translation Request, the TA performs the following basic steps:

1. Validates that the Function has been configured to issue ATS Translation Requests.
2. Determines whether the Function may access the memory indicated by the ATS Translation Request and has the associated access rights.
3. Determines whether a translation can be provided to the Function. If yes, the TA issues a translation to the Function.
 - a. ATS is required to support a variety of page sizes to accommodate a range of ATPT and processor implementations.
 - i. Page sizes are required to be a power of two and naturally aligned.
 - ii. The minimum supported page size is 4096 bytes. ATS capable components are required to support this minimum page size.
 - b. A Function must be informed of the minimum translation or invalidate size it will be required to support to provide the Function an opportunity to optimize its resource utilization. The smallest minimum translation size must be 4096 bytes.
4. The TA communicates the success or failure of the request to the RC which generates an ATS Translation Completion and transmits via a Response TLP through a RP to the Function.
 - a. An RC is required to generate at least one ATS Translation Completion per ATS Translation Request; i.e., there is minimally a 1:1 correspondence independent of the success or failure of the request.
 - i. A successful translation can result in one or two ATS Translation Completion TLPs per request. The Translation Completion indicates the range of translation covered.
 - ii. An RC may pipeline multiple ATS Translation Completions; i.e., an RC may return multiple ATS Translation Completions and these ATS Translation Completions may be in any order relative to ATS Translation Requests.
 - iii. The RC is required to transmit the ATS Translation Completion using the same TC (Traffic Class) as the corresponding ATS Translation Request.
 - b. The requested address may not be valid. The RC is required to issue a Translation Completion indicating that the requested address is not accessible.

When the Function receives the ATS Translation Completion, it either updates its ATC to reflect the translation or notes that a translation does not exist. The Function proceeds with processing its work request and generates subsequent requests using either a translated address or an untranslated address based on the results of the Completion.

- a. Similar to Read Completions, a Function is required to allocate resource space for each completion(s) without causing backpressure on the PCIe Link.
- b. A Function is required to discard Translation Completions that might be “stale”. Stale Translation Completions can occur for a variety of reasons.

As one can surmise, ATS Translation Request and Translation Completion processing is conceptually similar and, in many respects, identical to PCIe Read Request and Read Completion processing. This is intentional to reduce design complexity and to simplify integration of ATS into existing and new PCIe-based solutions. Keeping this in mind, ATS requires the following:

- ATS capable components must interoperate with [PCIe-1.1] compliant components.
- ATS is enabled through a new Capability and associated configuration structure. To enable ATS, software must detect this Capability and enable the Function to issue ATS TLP. If a Function is not enabled, the Function is required not to issue ATS Translation Requests and is required to issue all DMA Read and Write Requests with the TLP AT field set to “untranslated”.
- ATS TLPs are routed using either address-based or ID-based routing.
- ATS TLPs are required to use the same ordering rules as specified in this specification.
- ATS TLPs are required to flow unmodified through [PCIe-1.1] compliant Switches.
- A Function is permitted to intermix translated and untranslated requests.
- ATS transactions are required not to rely upon the address field of a memory request to communicate additional information beyond its current use as defined by the PCI-SIG.

IMPLEMENTATION NOTE: ADDRESS RANGE OVERLAP §

While significant overlap is expected, a system is not required to make Untranslated and Translation/Translated sequences interchangeable for all address ranges. For example, it is permitted for a Root Complex to require Untranslated Requests for interrupt behavior (MSI/MSI-X).

In contrast to the prior example, § Figure 10-3 illustrates an example Multi-Function Device. In this example Device, there are three Functions. Key points to note in § Figure 10-3 are:

- Each ATC is associated with a single Function. Each ATS-capable Function must be able to source and sink at least one of each ATS Translation Request or Translation Completion type.
- If an ATC is associated with a Physical Function implementing a Scalable I/O Virtualization Extended Capability structure, that ATC must either segregate or tag translations associated with each Scalable Device Interface.
- Each ATC is configured and accessed on a per Function basis. A Multi-Function Device is not required to implement ATS on every Function.
- If the ATC implementation shares resources among a set of Functions, then the logical behavior is required to be consistent with fully independent ATC implementations.

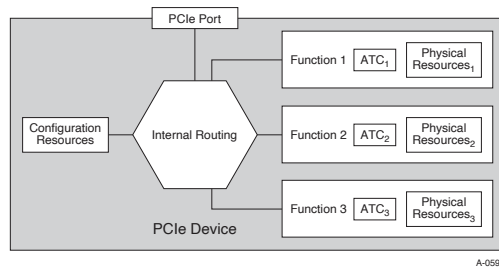


Figure 10-3 Example Multi-Function Device with ATC per Function §

Independent of the number of Functions within a Device, the following are required:

- A Function is required not to issue any TLP with the AT field set unless the address within the TLP was obtained through the ATS Translation Request and Translation Completion protocol.
- Each ATC is required to only be populated using the ATS protocol; i.e., each entry within the ATC must be filled via an ATS Translation Completion in response to the Function issuing an ATS Translation Request for a given address.
- Each ATC cannot be modified except through the ATS protocol. That is:
 - Host system software cannot modify the ATC other than through the protocols defined in this specification except to invalidate one or more translations in an ATC. A Device or Function reset would be an example of an operation performed by software to change the contents of the ATC, but a reset is only allowed to invalidate entries not modify their contents.
 - It must not be possible for host system software to use software executing on the Device to modify the ATC.

When a TA determines that a Function should no longer maintain a translation within its ATC, the TA initiates the ATS invalidation protocol. The invalidation protocol consists of a single Invalidate Request and one or more Invalidate Completions.

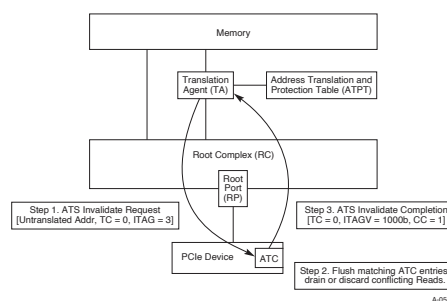


Figure 10-4 Invalidation Protocol with a Single Invalidate Request and Completion §

As § Figure 10-4 illustrates, there are essentially three steps in the ATS invalidation protocol:

1. The system software updates an entry in the tables used by the TA. After the table is changed, the TA determines that a translation should be invalidated in an ATC and initiates an Invalidate Request TLP which is transmitted from the RP to the example Single-Function Device. The Invalidate Request communicates an untranslated address range, the TC, and an RP unique tag which is used to correlate Invalidate Completions with the Invalidate Request.

2. The Function receives the Invalidate Request and invalidates all matching ATC entries. A Function is not required to immediately flush all pending requests upon receipt of an Invalidate Request. If transactions are in a queue waiting to be sent, it is not necessary for the Function to expunge requests from the queue even if those transactions use an address that is being invalidated.
 - a. A Function is required not to indicate the invalidation has completed until all outstanding Read Requests or Translation Requests that reference the associated translated address have been retired or nullified.
 - b. A Function is required to ensure that the Invalidate Completion indication to the RC will arrive at the RC after any previously posted writes that use the “stale” address.
3. When a Function has ascertained that all uses of the translated address are complete, it issues one or more ATS Invalidate Completions.
 - a. An Invalidate Completion is issued for each TC that may have referenced the range invalidated. These completions act as a flush mechanism to ensure the hierarchy is cleansed of any in-flight transactions which may contain references to the translated address.
 - i. The number of Completions required is communicated within each Invalidate Completion. A TA or RC implementation can maintain a counter to ensure that all Invalidate Completions are received before considering the translation to no longer be in use.
 - ii. If more than one Invalidate Completion is sent, the Invalidate Completion sent in each TC must be identical in the fields detailed in § Section 10.3.2.
 - b. An Invalidate Completion contains the ITAG from Invalidate Request to enable the RC to correlate Invalidate Requests and Completions.

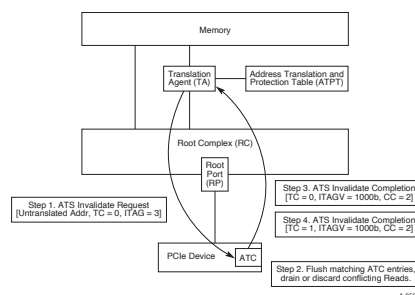


Figure 10-5 Single Invalidate Request with Multiple Invalidate Completions §

10.1.2 Page Request Interface Extension §

ATS improves the behavior of DMA based data movement. An associated Page Request Interface (PRI) provides additional advantages by allowing DMA operations to be initiated without requiring that all the data to be moved into or out of system memory be pinned.²⁰⁹ The overhead associated with pinning memory may be modest, but the negative impact on system performance of removing large portions of memory from the pageable pool can be significant.

PRI is functionally independent of the other aspects of ATS. That is, a device that supports ATS need not support PRI, but PRI is dependent on ATS's capabilities.

Intelligent I/O devices can be constructed to make good use of a more dynamic memory interface. Pinning will always have the best performance characteristics from a device's perspective—all the memory it wants to touch is guaranteed to be present. However, guaranteeing the residence of all the memory a device might touch can be problematic and force a sub-optimal level of device awareness on a host. Allowing a device to operate more independently (to page fault when it requires memory resources that are not present) provides a superior level of coupling between device and host.²¹⁰

209. Locked in place so that it cannot be swapped out by the system's dynamic paging mechanism.

The mechanisms used to take advantage of a Page Request Interface are very device specific. As an example of a model in which such an interface could improve overall system performance, let us examine a high-speed LAN device. Such a device knows its burst rate and need only have as much physical buffer space available for inbound data as it can receive within some quantum. A vector of unpinned virtual memory pages could be made available to the device, that the device then requests as needed to maintain its burst window. This minimizes the required memory footprint of the device and simplifies the interface with the host, both without negatively impacting performance.

The ability to page begs the question of page table status flag management. Typical TAs associate flags (e.g., dirty and access indications) with each untranslated address. Without any additional hints about how to manage pages mapped to a Function, such TAs would need to conservatively assume that when they grant a Function permission to read or write a page, that Function will use the permission. Such writable pages would need to be marked as dirty before their translated addresses are made available to a Function.

This conservative dirty-on-write-permission-grant behavior is generally not a significant issue for Functions that do not support paging, where pages are pinned and the cost of saving a clean page to memory will seldom be paid. However, Functions that support the Page Request Interface could pay a significant penalty if all writable pages are treated as dirty, since such Functions operate without pinning their accessible memory footprints and may issue speculative page requests for performance. The cost of saving clean pages (instead of just discarding them) in such systems can diminish the value of otherwise attractive paging techniques. This can cause significant performance issues and risk functional issues in circumstances where the backing store is unable to be written, such as a CD-ROM.

The No Write (NW flag in Translation Requests indicates that a Function is willing to restrict its usage to only reading the page, independent of the access rights that would otherwise have been granted.

If a device chooses to request only read access by issuing a Translation Request with the NW flag Set and later determines that it needs to write to the page, then the device must issue a new Translation Request.

Upon receiving a Translation Request with the NW flag Clear, TAs are permitted to mark the associated pages dirty. Functions *MUST@FLIT* not issue such Requests unless they have been given explicit write permission. An example of write permission is where the host issues a command to a Function to load data from a storage device and write that data into memory.

10.1.3 Process Address Space ID (PASID) §

Certain TLPs can optionally be associated with a Process Address Space ID (PASID). This value is conveyed using the PASID TLP Prefix (NFM) or OHC-A (FM). The PASID TLP Prefix is defined in § Section 6.20 .

PASID is permitted for the following types of TLPs:

- Memory Requests (including UIO, AtomicOp, and DMWr Requests) with Address Type (AT) of Untranslated or Translated
- Address Translation Requests (i.e., MRd with AT=01b)
- Page Request Messages
- ATS Invalidate Request Messages
- PRG Response Messages
- Address routed messages in Flit Mode

Usage of PASID for Untranslated Memory Requests is defined in § Section 6.20 . This section describes PASID for the remaining TLPs.

210. The alternative is a private interface between a device and its driver that is used to communicate device state so that the driver can ensure the availability of pinned memory resources.

Each Function has an independent set of PASID values. The PASID field is 20 bits wide, although typically the number of active PASID values will be significantly lower than 2^{20} . PASID values may be allocated sparsely. If the usable width is constrained by the TA or the ATC, the unused upper bits of the PASID value must be 0b. All $2^{\text{usable_width}}$ PASID values are usable. Function hardware is not permitted to assume there are “reserved” PASID values.

An ATC may optionally support Translated Requests with PASID. The Translated Requests with PASID feature is enabled when the Translated Requests with PASID Enable bit is Set. When Translated Requests with PASID Enable is Set, an ATC is permitted to issue Translated Requests with a PASID. When enabled, if the ATC obtained a translation using a Translation Request with PASID, the corresponding Translated Request must carry the same PASID as the Translation Request, the Privileged Mode Requested bit must match the Privileged Mode Access bit in the Translation Response, and the Execute Requested bit is permitted to be Set only if the Execute Permitted bit in the Translation Response was Set. Similarly, if the ATC obtained a translation using a Translation Request without a PASID, the corresponding Translated Request must not carry a PASID. If these rules are not followed the resulting system behavior is undefined.

10.1.4 ATS Memory Attributes §

ATS Memory Attributes (AMAs) provide hints for performing memory operations such as cache management. The AMAs may be supplied to an Endpoint device by a Translation Completion and stored in the ATC. If the ATS Memory Attributes Enable bit is Set, the Endpoint function retrieves the AMAs from the ATC and is permitted to provide AMAs with Memory Read/Write TLPs using the TPH TLP Prefix. This serves as a performance optimization by preventing the TA from having to perform a look-up. The TA and ATC are permitted to support AMAs. If the ATC does not support AMAs but the TA does, then the TA is permitted to perform a lookup to obtain AMAs when processing Memory Read/Write TLPs received from the Endpoint device. A TA that supports AMAs is permitted to always return AMAs in all ATS Translation Completions to all Endpoints.

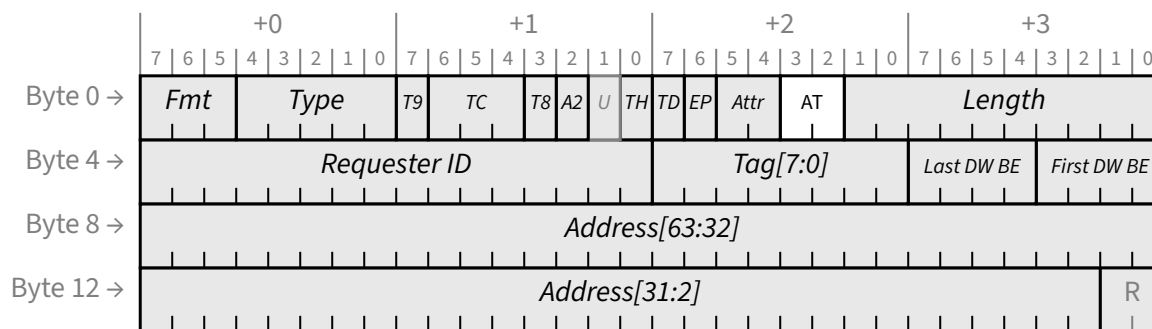
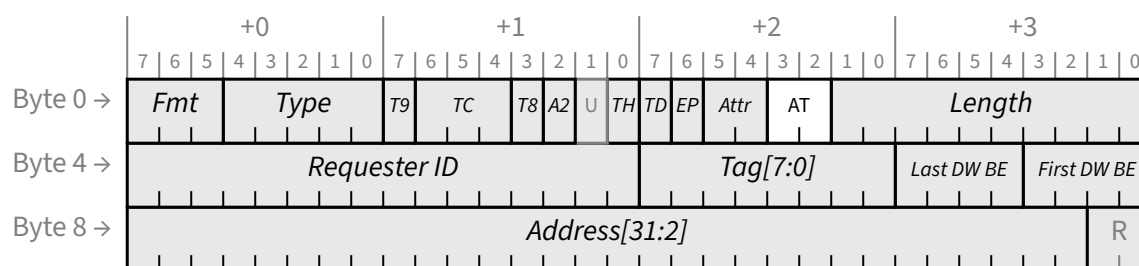
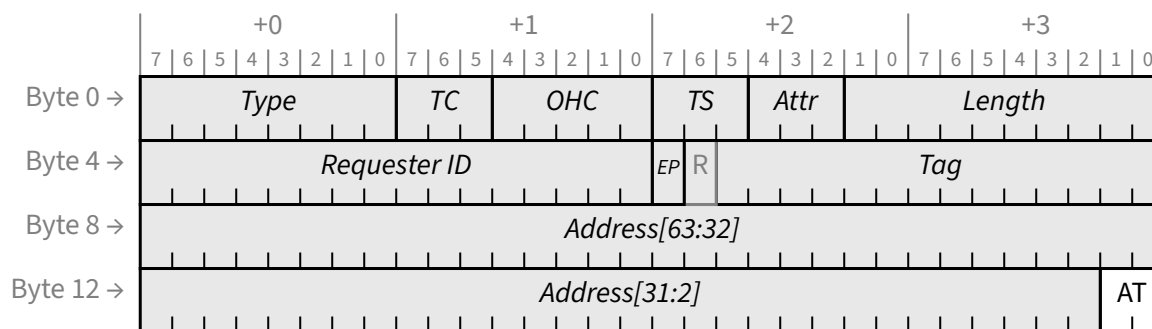
10.2 ATS Translation Services §

A TA does translations. An ATC can cache those translations. If an ATC is separated from the TA by PCIe, the memory request from an ATC will need to be able to indicate if the address in the transaction is translated or not. The modifications to the memory transactions are described in this section, as are the transactions that are used to communicate translations between a remote ATC and a central TA.

When Shadow Functions are enabled, the TA must treat identically all Requests from the “main” Function and its Shadows (See § Section 7.9.25). This usually involves ensuring that system software configures the translation tables used by the TA provide identical answers for the “main” Function and its Shadows. Depending on the architecture of a specific TA, more work may be required (e.g., managing dirty bits, ensuring caches are consistent, etc.). Such work is outside the scope of this specification.

10.2.1 Memory Requests with Address Type §

A Function with an ATC can send Memory Requests that contain either translated or untranslated addresses. The Address Type (AT) field is used to indicate the type of address that is present in the request header (see § Figure 10-6, § Figure 10-7, § Figure 10-8, and § Figure 10-9).

Figure 10-6 Memory Request Header with 64-bit Address Highlighting *AT* field §Figure 10-7 Memory Request Header with 32-bit Address Highlighting *AT* field §Figure 10-8 Memory Request Header with 64-bit Address Highlighting *AT* field - Flit Mode §

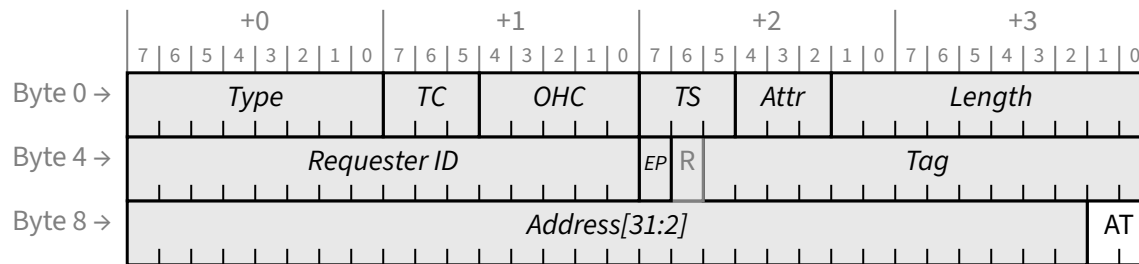


Figure 10-9 Memory Request Header with 32-bit Address Highlighting AT field - Flit Mode §

In NFM, the AT field in the Requests is a redefinition of a reserved field in earlier version of this specification.

Functions that do not implement an ATC will continue to set the AT field to its defined reserved value (00b). Functions that implement an ATC will set the AT field as listed in § Table 10-1.

Table 10-1 Address Type (AT) Field Encodings §

AT[1:0] Coding	Mnemonic	Meaning
00b	Untranslated	A TA may treat the address as either virtual or physical.
01b	Translation Request	The TA will return the translation of the address contained in the address field of the request as a read completion. This value only has meaning for an explicit Translation Request (see § Section 10.2.2). The TA will signal an Unsupported Request (UR) if it receives a TLP with the AT field set to 01b in a Memory Request other than Memory Read.
10b	Translated	The address in the transaction has been translated by an ATC. If the Function associated with the SourceID is allowed to present physical addresses to the system memory, then the TA might not translate this address. If the Function is not allowed to present physical addresses, then the TA may treat this as an UR.
11b	Reserved	The TA will signal an Unsupported Request (UR) if it receives a Memory Request TLP with the AT field set to 11b.

The AT field is only defined for Memory Requests and in Flit Mode, Address Routed Messages. The field remains reserved for other TLPs.

10.2.2 Translation Requests §

A Translation Request has a format that is similar to that of a Memory Read²¹¹ (MRd TLP Type) in either FM or NFM. The AT field must be set to the value defined for "Translation Request" to differentiate a Translation Request from a normal Memory Read (see § Figure 10-10, § Figure 10-11, § Figure 10-12, and § Figure 10-13). In Flit Mode, OHC-A1 must be included, and the No Write (NW) flag is contained in OHC-A1).

Translation Requests have the same completion timeout intervals as Read Requests.

211. Translation Requests are not defined for UIO Memory Reads (UIOMRd TLP Type).

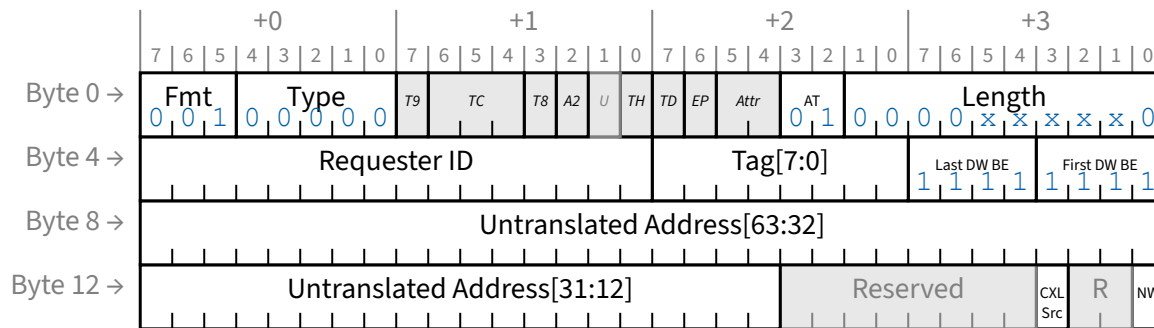


Figure 10-10 Translation Request with 64-bit Address - Non-Flit Mode §

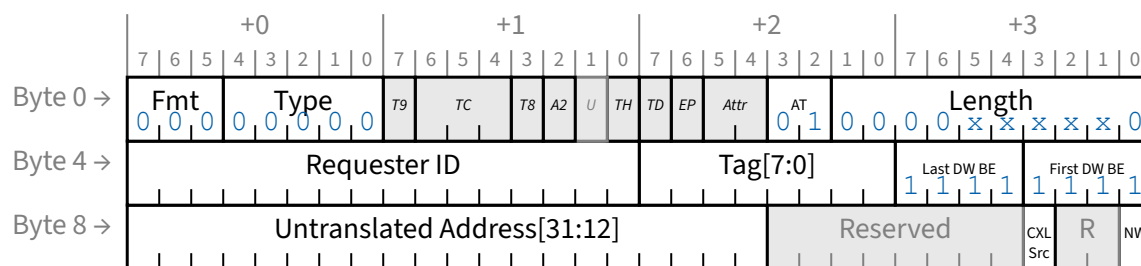
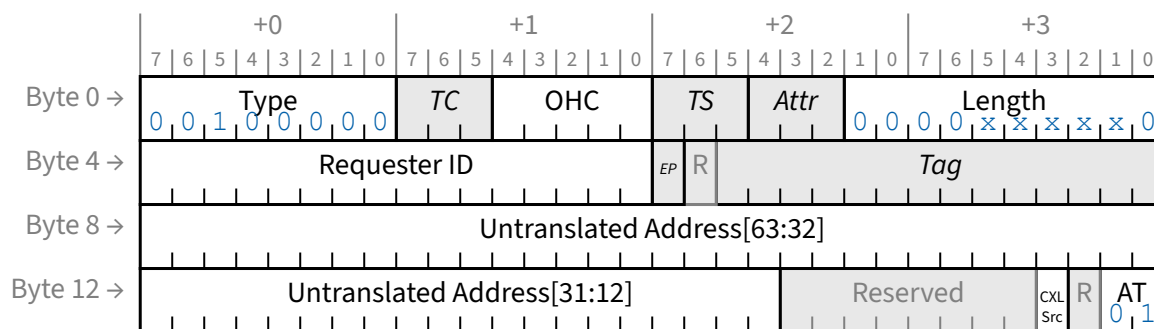


Figure 10-11 Translation Request with 32-bit Address - Non-Flit Mode §

Figure 10-12 Translation Request with 64-bit Address - Flit Mode ²¹² §

212. The NW bit is located in OHC-A1.

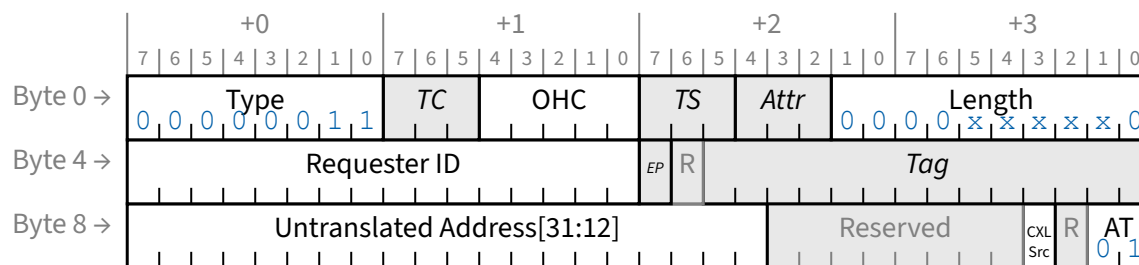


Figure 10-13 Translation Request with 32-bit Address - Flit Mode²¹³ §

10.2.2.1 Attribute Field §

For a Translation Request, the Relaxed Ordering (RO) bit is applicable and permitted to be Set, where it affects the ordering of its associated Translation Completions. The remainder of the Attr field is Reserved. The Requester of a Translation Request must not depend on the TA to guarantee any specific ordering relationship between Translation Completions and any other Requests or Completions. There are no ordering requirements for a Translation Request. A TA may reorder a Translation Request with respect to any other request.

IMPLEMENTATION NOTE: TRANSLATION REQUEST ORDERING §

Because no ordering can be assumed between Translation Requests and other types of Requests, a Translation Request does not make an effective flushing/ordering primitive.

10.2.2.2 Length Field §

The Length field is set to indicate how many translations may be returned in response to this request. Each translation is 8 bytes in length and represents one or more STUs (Smallest Translation Unit). The maximum setting for the Length field is the RCB of the Root Port as determined by Read Completion Boundary (RCB) in the Link Control Register. The Length field in a Translation Request must always indicate an even number of DWORDs. If Length is set to indicate a value greater than allowed, or if the least-significant bit of the Length field is non-zero, then the resulting handling by the TA is undefined.²¹⁴

If the Length field has a value greater than two, then the Function is requesting translations for a range of memory greater than a single STU. The additional translations, if provided, are assumed to be for sequentially-increasing, equal-sized, STU-aligned regions, starting at the requested address.

10.2.2.3 Tag Field §

The Tag field has the same meaning as in a Memory Read Request.

213. The NW bit is located in OHC-A1.

214. Due to ambiguous text in earlier versions of this document, it should be assumed that an unrecoverable error condition may occur.

10.2.2.4 Untranslated Address Field §

A Translation Request includes either a 32-bit or a 64-bit Untranslated Address field. This field indicates the address to be translated. The TA will make decisions about the validity of the request, based on the address in the translation request. The TA is permitted to return fewer translations than requested, but it will not return more.

When multiple translations are requested, the TA will not return a translation if the range of that translation does not overlap the implied range of the Translation Request (this would only apply to translations after the initial value). The implied range of the Translation Request is $[2^{\text{STU}+12} * (\text{Length}/2)]$ bytes.

The Untranslated Address field in the Translation Request is any address in the range of the first STU. Address bits 11:0 are not present in the Translation Request and are implied to be zero. If a Requester has Page Aligned Request Set (see § Section 7.8.9.2), it must ensure that bits 11:2 are zero. If a Requester has Page Aligned Request Clear, it is permitted to supply any value for bits 11:2.²¹⁵ The TA must ignore bits 11:2 as well as any low-order bits not required to determine the translation.

For example, if using 64-bit addressing for a Function with the Page Aligned Request bit Set that is programmed with an STU of 1 (i.e., 8192-byte pages), bits 63:13 are significant, bit 12 is ignored by the TA and bits 11:0 are implied to be zero.

10.2.2.5 No Write (NW) Flag §

The No Write flag, when Set, indicates that the Function is requesting read-only access for this translation.²¹⁶

The TA may ignore the No Write Flag, however, if the TA responds with a translation marked as read-only then the Function must not issue Memory Write transactions using that translation. In this case, the Function may issue another translation request with the No Write flag Clear, which may result in a new translation completion with or without the W (Write) bit Set.

Upon receiving a Translation Request with the NW flag Clear, TAs are permitted to mark the associated pages dirty. Functions *MUST@FLIT* not issue such Requests unless they have been given explicit write permission.

In Flit Mode, the NW bit is part of OHC-A1.

10.2.2.6 PASID on Translation Request §

If a Translation Request has a PASID, the Untranslated Address Field is an address within the process address space indicated by the PASID field.

If a Translation Request has a PASID with either the Privileged Mode Requested or Execute Requested bit Set, these may be used in constructing the Translation Completion Data Entry.

The PASID Extended Capability indicates whether a Function supports and is enabled to send and receive TLPs with the PASID.

10.2.2.7 CXL Src §

This bit is assigned for use by [CXL]. In non-CXL systems, this bit is Reserved.

215. Note: The Page Aligned Request bit was added in Revision 1.1 of the ATS Specification.

216. Note: The No Write Flag was added in Revision 1.1 of the ATS Specification.

10.2.3 Translation Completion §

A Translation Completion (either a Cpl or a CplD) is sent by a TA for each Translation Request. This specification describes the meaning of fields in Translation Completions. Fields not defined in this chapter have the same meanings proscribed for Read Completions in Chapter 2. For a Translation Completion, the Relaxed Ordering (RO) bit is applicable and permitted to be Set if the corresponding Translation Request RO bit was set. The remainder of the Attr field is Reserved.

If the TA was not able to perform the requested translation, a Completion with no data (Cpl) must be returned.

IMPLEMENTATION NOTE: BYTE COUNT FIELD FOR UNSUCCESSFUL TRANSLATION COMPLETIONS WITH NO DATA §

Previous versions of this specification indicated the Byte Count and Lower Address field should be 0000 0000 0000b for Unsuccessful Translation Completions with No Data. It is strongly recommended that implementations do not depend on the Byte Count and Lower Address field being set to any particular value in Unsuccessful Translation Completions with No Data.

The values and meaning for the Completion Status field are listed in § Table 10-2, where return values other than *Success* indicate an error.

Table 10-2 Translation Completion Status Codes §

Value	Status	Meaning
000b	Success	Indicates that the TA is returning one or more Translation Completion entries (including entries with R=W=0b).
001b	Unsupported Request (UR)	Indicates there is a problem, potentially correctable by system software, that prevents the TA from returning any Translation Completion entries. It is permitted for a Function that receives this Completion code to disable its ATC and not send requests using translated addresses until the ATC is re-enabled. The mechanism a Function receiving this code uses to report this condition is outside the scope of this specification. The TA detecting this error is a "Completer Sending a Completion with UR/CA Status" and shall behave as defined in this specification.
010b	RRS	This value is not allowed in any Completion to a Translation Request initiated by a PCI Express Function. If received by a Function, it is permitted to be handled as a Malformed TLP.
100b	Completer Abort (CA)	The TA was not able to translate the address because of an error in the TA. This status indicates a more serious error condition than UR. Returning CA nominally causes an error to be reported to the device driver associated with the ATC. See AER in this specification.
All others	Reserved	A Translation Completion with a Reserved Completion Status value is treated as if the Completion Status was Unsupported Request (001b).

Completion header fields must be set in accordance with § Section 2.2.9 and § Section 2.3 .

Translation Completions must be sent using the same TC as the Translation Request. The Function is not required to verify that the same TC was used. A TA may optionally copy the RO bit of a Translation Request to the Translation Completion in accordance to the rule specified for the attribute field of completions in § Section 2.2.9 . It is strongly

recommended that the TA copy the RO bit. However, if a TA does not copy the RO bit of a Translation Request to the Translation Completion, the TA must clear the RO bit in the Translation Completion.

A Translation Completion with RO Set must follow the ordering rules for Relaxed Ordered Completions as specified in § Section 2.4.1 . A Function that initiates a Translation Request with the RO bit Set but receives the associated Translation Completion with the RO bit Clear is permitted to order associated Translation Completions as if the RO bit is Set.

A Function that initiates a Translation Request with the RO bit Set, must not report an error if the associated Translation Completion has the RO bit Clear.

IMPLEMENTATION NOTE: ATTRIBUTE FIELD COMPATIBILITY IN TRANSLATION COMPLETIONS §

Some implementations of TA may not copy the Attribute field from the request to the completion as required by § Section 2.2.9 , since the Attr field is defined as Reserved in previous versions of this specification. Therefore, the following situations may occur and are handled as follows:

- A TA that does not copy the RO bit (as is typically done for completions as indicated in § Section 2.2.9) by forcing RO to 0 is coupled with a Function conforming to this specification that allows RO to be Set in the request. The Function will accept a Translation Completion with RO Clear and not log an error.
- A TA that conforms to the Attr copy rule (in § Section 2.2.9) is coupled with a Function that does not support RO in Translation Requests. Translation Completions will return with RO Clear as the Function expects.

Therefore, the use of RO is made fully backward compatible. However, it is strongly recommended that the TA support the copy of the RO bit conforming to the rules in § Section 2.2.9 .

The Lower Address field contains a computed value that makes the packet consistent with RCB semantics. If the result is returned in a single packet, Lower Address contains RCB minus Byte Count. If the results are returned in two packets, Lower Address for the first packet contains RCB minus (Total Completion Length * 4). Lower Address for subsequent packets contains 000 0000b. See § Section 10.2.4 for additional requirements for multiple packet completions.

If the Completion Status field is 000b, then the translation was successful and a data payload will follow the header. The contents of the data payload are shown in § Figure 10-14.

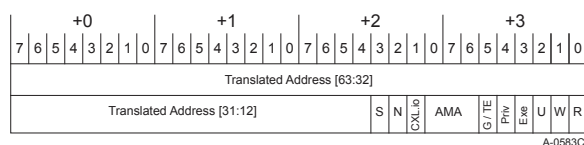


Figure 10-14 Translation Completion Data Entry §

Table 10-3 Translation Completion Data Fields §

Field	Meaning
S	Size of translation - This field is 0b if the translation applies to a 4096-byte range of memory. If this field is 1b, then the translation applies to a range of memory that is larger than 4096 bytes (see § Section 10.2.3.1).

Field	Meaning				
N	Non-snooped accesses - If this field is 1b, then the read and write requests that use this translation must Clear the No Snoop bit in the Attribute field. If it is 0b, then the Function may use other means to determine if No Snoop should be Set.				
CXL.io	This bit is assigned for use by [CXL]. In non-CXL systems, this bit is Reserved.				
AMA	ATS Memory Attributes (AMAs) – If the ATS Memory Attributes Supported bit is Set, the ATC is permitted to cache this AMA value. <table> <tr> <th>AMA</th><th>ATS Memory Attributes</th></tr> <tr> <td>000b - 111b</td><td>ATS Memory Attribute values (implementation specific). The default value is 000b.</td></tr> </table>	AMA	ATS Memory Attributes	000b - 111b	ATS Memory Attribute values (implementation specific). The default value is 000b.
AMA	ATS Memory Attributes				
000b - 111b	ATS Memory Attribute values (implementation specific). The default value is 000b.				
G or TE	Global Mapping (G) – If this bit is Set, the ATC is permitted to cache this mapping entry in all PASIDs. If Clear, the ATC is permitted to cache this mapping entry only in the PASID associated with the requesting PASID. This bit may only be Set if the associated Translation Request had a PASID. The presence of this field is mutually exclusive with the TEE Exclusive Memory Attribute (TE) field. TEE Exclusive Memory Attribute (TE) – If this bit is Set, the Translated Addresses are associated with TEE memory. If Clear, the Translated Addresses are associated with non-TEE memory (see § Section 10.2.3.10). The presence of this field is mutually exclusive with the Global Mapping (G) field. If the Global Invalidate Supported is Set in the <u>ATS Capability Register</u>, this field must be treated as the Global Mapping (G) field (see § Section 10.2.3.8). If the TEE Exclusive Memory Attribute Supported is Set in the <u>ATS Capability Register</u>, this field must be treated as the TEE Exclusive Memory Attribute (TE) field (see § Section 10.2.3.10). If neither Global Invalidate Supported nor TEE Exclusive Memory Attribute Supported is Set in the <u>ATS Capability Register</u>, this field must be treated as Reserved.				
Exe	Execute Permitted - If this bit is Set, the requesting Function is permitted to execute code contained in the associated memory range. This bit may be Set only if the associated Translation Request had the <u>Execute Requested</u> bit Set. If this bit is Set, R must also be Set. The Priv bit indicates the Privilege level associated with the Exe bit. If Priv is Set, the Exe bit indicates permissions associated with Privileged Mode entities in the Function. If Priv is Clear, the Exe bit indicates permissions associated with Non-Privileged Mode entities in the Function. This value may be cached if R is Set.				
Priv	Privileged Mode Access - If this bit is Set, R, W and Exe refer to permissions associated with Privileged Mode entities. If this bit is Clear, R, W and Exe refer to permissions associated with Non-Privileged Mode entities. This bit may only be Set if the associated Translation Request had the <u>Privileged Mode Requested</u> bit Set. This value must be cached any of the R, W or Exe values are cached.				
U	Untranslated access only - When this field is Set, the indicated range may only be accessed using untranslated addresses, and the Translated Address field of this Translation Completion Data Entry may not be used in a subsequent Read/Write Request with <u>AT</u> set to Translated. This value may be cached if R or W is Set. Future revisions of this specification are expected to deprecate the U bit.				

Field	Meaning
R,W	Read, Write - These two fields indicate the transaction types that are allowed for requests using the translation. The encodings are:
	00b Neither read nor write transactions are allowed. This translation is considered not to be valid. The contents of the Translated Address, N, AMA, TE, U, and Exe fields are undefined. A translation with this value must not be cached in the ATC (see § Section 10.2.3.5).
	01b Write Requests that target this range are allowed, but Read Requests are not unless they are zero-length reads.
	10b Read Requests that target this range are allowed (including zero-length reads), but Write Requests are not.
	11b Read and Write Requests that target this range are allowed.
	The Priv bit indicates the Privilege level associated with R and W. If Priv is Set, R and W indicate permissions associated with Privileged Mode entities in the Function. If Priv is Clear, R and W indicate permissions associated with Non-Privileged Mode entities in the Function.
	If AMA is enabled (AMAE bit is 1b), a zero-length Read Request using a translated address must include AMAs in the TPH TLP Prefix.

10.2.3.1 Translated Address Field §

If the R and W fields are both Clear, or if U is Set, then the Translated Address field may not be used by the Function for any purpose.

If either the R or W field is Set, and the U field is Clear, then the Translated Address field contains an address that can be used by the Function in a Memory Request with the AT field set to Translated and the Function may cache the Translated Address. When cached, the R and W fields must be stored with the same value as the Translation Completion entry. The address that is cached must be a subset of the address range indicated in the Translation Completion (the subset may include the entire range).

While the Translated Address is cached in the Function's ATC, it shall not be possible for the Function to modify the entry other than to delete it. The entry must be deleted from the ATC when an Invalidate Request is received that has an indicated range that overlaps any portion of the cached address.

A Function is not allowed to make an entry into its ATC unless the entry is in a Translation Completion and the E (Enable) field within the ATS Capability is Set. Entries in an ATC cache that are written before the E field is Set must not be used in Memory Request. They must either be invalidated when the E field is Set or ignored and not used.

10.2.3.2 Translation Range Size (S) Field §

If S is Set, then the translation applies to a range that is larger than 4096 bytes. If S = 1b, then bit 12 of the Translated Address is used to indicate whether or not the range is larger than 8192 bytes. If bit 12 is 0b, then the range size is 8192 bytes, but it is larger than 8192 bytes if Set. If S = 1b and bit 12 = 1b, then bit 13 is used to determine if the range is larger than 16384 bytes or not. If bit 13 is 0b, then the range size is 16384 bytes, but it is larger than 16384 bytes if Set.

Low-order address bits are consumed in sequence to indicate the size of the range associated with the translation.

Note: This encoding method is also used to indicate the size of the memory range being invalidated.

Examples for different translation sizes are shown in § Table 10-5.

Table 10-5 Examples of Translation Size Using S Field §

Address Bits																				S	Translation Range Size in Bytes	
63:32 *	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13			12
x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	0	4 K
x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	0	1	8 K
x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	x	0	1	1	16 K
x	x	x	x	x	x	x	x	x	x	x	x	0	1	1	1	1	1	1	1	1	1	2 M
x	x	x	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1 G
x	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	4 G

Note:

* Upper address bits are used to indicate the size for ranges larger than 4 GB.

The size field is set to indicate the range size in multiples of 4096 bytes regardless of the setting of STU. For example, if STU is set to indicate that the minimum translation is 8192 bytes, then S should be Set on all translation returned in a Translation Completion and in all Invalidate Requests. If STU is set to indicate a 16384-byte minimum, then S and bit 12 would both be Set in all translation and invalidate ranges.

If S is Set and bits 63:12 are all 1b, then the behavior is undefined. If S is Set and bit 63 is 0b, and bits 62:12 are all 1b, then the request is to invalidate all translations.

If a Function receives a Translation Completion with a Translation Size field smaller than the Function's programmed STU value, it shall treat the Translation Completion as if it had Completion Status UR.

10.2.3.3 Non-snooped (N) Field §

This field is Set to indicate that Read and Write Requests that target memory in the range of this translation must Clear the No Snoop Attribute bit in the Request header. When this field is 0b, the Function is allowed to Set the No Snoop Attribute bit in a Function-specific manner.

Note: When this field is Cleared, the Function is not allowed to Set No Snoop in a Memory Request if the Enable No Snoop field in the Device Control register is Cleared.

The N bit may be cached by the ATC if either R or W is Set.

When U is Set, the meaning of this field is undefined, and the TA may set this field to any value. A translation has a single value for the N field that is not affected by privilege level. An ATC is permitted to cache the N field without regard to the value of the Priv bit.

10.2.3.4 Untranslated Access Only (U) Field §

Future revisions of this specification are expected to deprecate the U bit.

This field is Set when the Function is not allowed to access the implied range of memory using a translated address (the range is implied by the untranslated address in the Translation Request and the offset of the translation in the Translation Completion). The Function may use untranslated addresses to access the range as long as the accesses are allowed by the R and W fields. The Function may cache this translation value if either R or W is Set. If the U field is Set, the Translated Address field in the translation is not necessarily a valid memory address and the Function may not use the value in a Read or Write Request with AT set to Translated.

Note: One of the possible uses of this field is to avoid unnecessary invalidations. If a Function uses translated requests for some portions of memory, but not others, then the U field can be used on the portions for which translated requests are not used. When a translation changes if the U field is Set, then it will not necessarily be required that an Invalidate Request be sent to the Function. An example of this use is a Function with a ring buffer that is used for commands. The ring buffer may be allocated for a long period of time and have very high reuse (locality). For this reason, it is useful for the Function to use translated addresses in its memory request that target the command buffer. The same Function might access data buffers that have poor locality and low reuse. Accesses to the data buffers might best be handled by using untranslated Requests. Setting the U field for the data buffer translations ensures that the Function will not attempt to use a translated value to access the data buffer so, when the data buffer mappings are changed, no Invalidate Request is required. When U is Set and either R or W is Set, the ATC is permitted to cache U, R, W Exe, and Priv, as well as the Translation Range Size (see § Section 10.2.3.2). An Invalidate Request is required if these values change.

10.2.3.5 Read (R) and Write (W) Fields §

These fields indicate if the returned translation value may be used in a read or write memory request. The ATC is not permitted to issue a non-zero length read request using the translation value if the R field is Cleared. The ATC is not permitted to issue a write request using the translation value if the W field is Cleared. The ATC may not issue any type of request using the translation value if neither the R nor W fields are Set. If both R and W fields are Cleared, the range of the translation is still indicated, but the meaning of the other values in the translation is undefined.

Note: The range of a Translation entry is indicated even if R = W = 0b in order to allow a “hole” in the Translation Completion. For example, if the Translation Request has a Length of six DWs, then up to three translations could be included in the Translation Completion. The first and third translations may have Set R or W but the second could have R = W = 0b. To avoid ambiguity about the size of the indicated gap, the range of the gap is indicated in the Translation Completion even if R = W = 0b.

The R = 0b, W = 0b state is used to indicate that the ATC is not permitted to use address field in the translation to form a translated address value for a subsequent request.

When the host changes a translation in the TA from not valid to valid, the host is not required to send an invalidation indication to the ATC. As such the ATC will not be notified of such changes and thus the ATC is not permitted to cache a translation value of R = W = 0b.

When the host changes permissions associated with a translation in the TA, to grant additional permission (e.g., R = 1, W = 0b to R = W = 1b), the host is not required to send an invalidation indication to the ATC. As such, the ATC will not be notified of such changes. When the ATC requires permissions greater than the cached ATC entry, the ATC is permitted to request a fresh translation.

When the host changes permissions associated with a translation in the TA, to remove permission (e.g., R = W = 1b to R = 1b, W = 0b), the host is required to send an invalidation indication to the ATC. The subsequent Invalidate Completion tells the host that the ATC has stopped using the previously granted permissions.

If no table entry is found for the requested address, the TA must return a CplID with a single translation value with R = W = 0b.

Note: Implementations should not assume that receiving a translation response with the R or W bits Set (*independent of the value of the U bit*) implies that a subsequent read or write request with the same **untranslated** address will succeed. Although it may be possible for a device and its controlling software to ensure this property, the method for doing so is outside the scope of this specification.

When ACS Direct Translated P2P is enabled, Translated and Untranslated requests may take different paths between Requester and Completer. As such, PCI Express Ordering between Translated and Untranslated requests is not guaranteed. For transaction sequences that require ordering, Functions should avoid using a mixture of Translated and Untranslated Requests.

The Priv bit is used to qualify R and W. If Priv is Set, R and W indicate permissions granted to Privileged Mode entities in the Function. If Priv is Clear, R and W indicate permissions granted to Non-Privileged Mode entities in the Function. The R and W values for the two privilege levels are independent. The ATC must not assume any correlation between the Privileged Mode and Non-Privileged Mode permissions associated with a translation.

10.2.3.6 Execute Permitted (Exe) §

If Exe is Set, the requesting Function is permitted to execute code in the implied range of memory. If Exe is Clear, the requesting Function is not permitted to execute code in the implied range of memory.

The definition of what it means for a Function to execute code is outside the scope of this specification. Various system components may have different instruction sets. Behavior within the requesting Function when it attempts to execute code that is not permitted by this bit is outside the scope of this specification.

The Exe bit may only be Set if the TA supports Execute permissions, the associated Translation Request had an effective value of 1b for the Execute Requested bit²¹⁷ and R is Set in the Translation Completion Data Entry. Otherwise, the Exe bit must be Clear.

This value may be cached if R is Set.

The Priv bit is used to qualify the Exe bit. If Priv is Set, the Exe bit indicates permission granted to Privileged Mode entities in the Function. If Priv is Clear, the Exe bit indicates permission granted to Non-Privileged Mode entities in the Function. The Exe bit values for the two privilege levels are independent. The ATC must not assume any correlation between the Privileged Mode and Non-Privileged Mode permissions associated with a translation.

Functions may optionally check that:

- If the Execute Requested bit is Clear in a Translation Request, the Exe bits in the associated Translation Completion Data Entries are also Clear.
- If Exe is Set, R is also Set.

If either optional check fails, the Function shall signal Unexpected Completion (UC). These checks are independently optional.

10.2.3.7 Privileged Mode Access (Priv) §

If Priv is Set, R, W, and Exe refer to permissions granted to entities operating in Privileged Mode in the requesting Function. If Priv is Clear, R, W, and Exe refer to permissions granted to entities operating in Non-Privileged Mode in the requesting Function.

The meaning of Privileged Mode and Non-Privileged Mode and what it means for an entity to be operating as in Privileged Mode or in Non-Privileged Mode depends on the protection model of the system and is outside the scope of this specification.

Behavior is outside the scope of this specification when an entity in the requesting Function attempts to access memory that it is not permitted to access.

The Priv bit may only be Set if the TA supports Privileged Mode and the associated Translation Request had a PASID TLP Prefix (NFM) or OHC-A1 with an effective value of 1b for the Privileged Mode Requested²¹⁸ bit. Otherwise, the Priv bit must be Clear.

217. The effective value of the bit is 0b unless the bit in the PASID TLP Prefix (NFM) or OHC-A1 is 1b and usage of the bit is enabled for the request.

218. The effective value of the bit is 0b unless the bit in the in the PASID TLP Prefix (NFM) or OHC-A1 is 1b and usage of the bit is enabled for the request.

The Privileged and Non-Privileged Mode versions of R, W and Exe are independent. An ATC may cache either or both versions of R, W and Exe. An ATC that receives a translation with R=W=0b for one privilege level may not assume anything about what it might receive for the other privilege level.

This value may be cached if R or W is Set. This value must be cached when the corresponding R, W, or Exe values are cached.

Note: Since the Priv bit is Set only when the requesting Function Sets the Privileged Mode Requested bit, Functions that never set that bit should always receive the Priv bit Clear and thus don't need to cache it.

Functions may optionally check that when the Privileged Mode Requested bit is Clear in a Translation Request, the Priv bits in the associated Translation Completion Data Entries are also Clear. If this optional check fails, the Function shall signal Unexpected Completion (UC).

IMPLEMENTATION NOTE: EXECUTE PERMISSION AND PRIVILEGE MODE ENFORCEMENT §

The requesting Function determines whether a particular Memory Request needs Execute permission or is associated with a Privileged Mode or Non-Privileged Mode entity. The ATC implements the protection checks indicated by the Exe and Priv bits.

10.2.3.8 Global Mapping (Global) §

If Global is Set, the requesting Function is permitted to create a Global Mapping entry in the ATC for this translation. If Global is Clear, the requesting Function is not permitted to create a Global Mapping entry in the ATC for this translation. Global Mapping entries apply to all PASIDs of the Function. They permit the ATC to reduce the number of translation requests needed and to reduce the memory needed for caching the results.

A Function is permitted to ignore this bit and always create non-Global Mapping entries in the ATC. This could result in multiple translations being requested for the same Untranslated Address under different PASIDs.

Functions that use this bit must also have the Global Invalidate Supported bit Set (see § Section 10.5.1.2).

IMPLEMENTATION NOTE: TLP LENGTH, ADDRESS, AND BYTE OFFSET VALUES FOR TRANSLATION REQUEST COMPLETIONS §

The intention behind the rules for Translation Completions containing multiple translations is to make the TLPs look similar to those contained in a Memory Read Completion.

For single TLP Translation Completions, the goal is to make the Completion look as though it is a Memory Read Completion that ends on an RCB. As such, the Length and Byte Count will indicate the same value, and the Lower Address will contain the value that makes the TLP appear to end on an RCB.

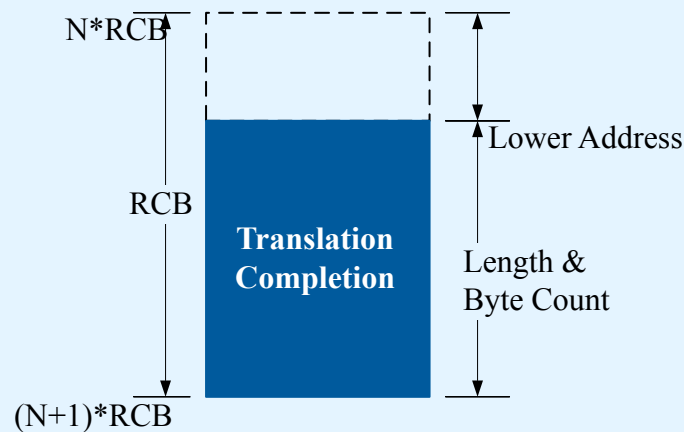


Figure 10-15 Example Translation Completion with 1 TLP §

To intermediate components (which do not track the transaction) this TLP will be indistinguishable from a Memory Read Completion ending on an RCB.

For Translation Completions consisting of two TLPs, the goal is to make the Completion look as though it is a Memory Read Completion that crosses an RCB. As Such, the first Completion TLP will contain Lower Address & Length values which make the TLP appear to end on an RCB. The Byte Count of the first TLP will indicate the total length of all the Translation Completions sent in this transaction. For the second TLP, the Length and Byte Count fields will indicate the same value, and the Lower Address value will be 0.

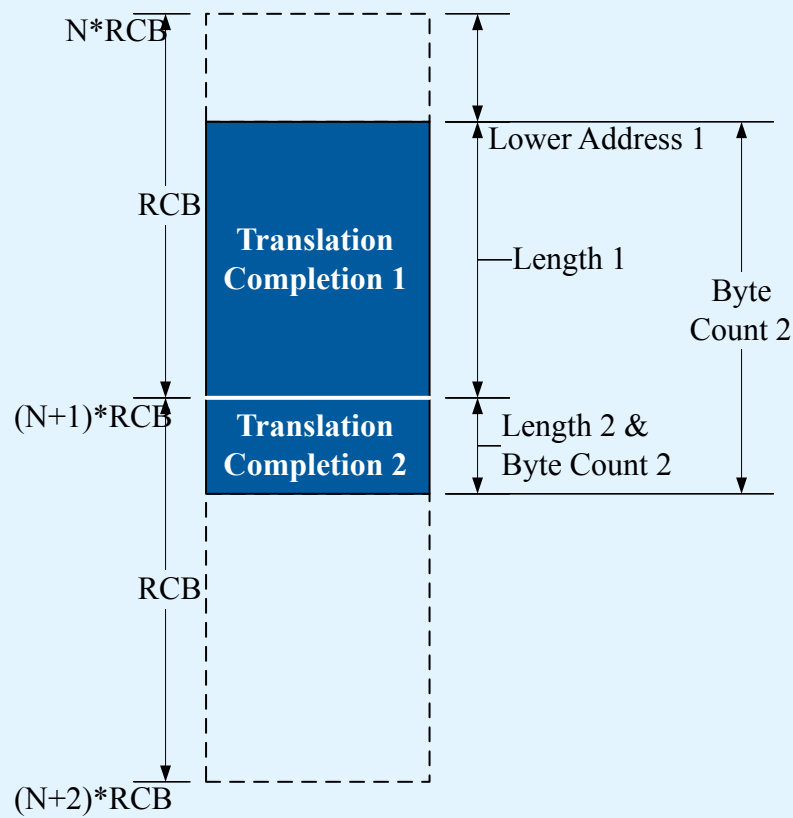


Figure 10-16 Example Translation Completion with 2 TLPs §

To intermediate components (which do not track the transaction) this Completion transaction will be indistinguishable from a Memory Read Completion that crosses an RCB.

Note that the Length field is measures DWORDs, whereas the Lower Address and Byte Offset fields are measured in Bytes.

Provided the TLPs are properly formatted, A TA may choose to split the Translation Completion between any 2 Translation Completion Data Entries. Because an ATC cannot request more translations than can fit within a single RCB, the architectural maximum number of Translation Completion Data Entries can be sent in a single Completion TLP.

10.2.3.9 ATS Memory Attributes §

AMA values are associated with the Completer and are implementation specific. AMA values are opaque to the Requester, the Requester's ATC and the TA.

The method for detecting support of, and for enabling, the AMA mechanism in a TA is implementation specific. When the AMA mechanism is not enabled in both the Requester and the TA, the default AMA value is 000b. Behavior of the Completer for this case is implementation specific.

An ATC that supports AMA, caches AMA values along with other ATC information (Translated Address, Read, Write permission etc.) If the AMA value changes, for a page that could be cached in an ATC, software must issue an ATS Invalidate Request covering that page to ensure the ATC flushes the stale AMA value.

IMPLEMENTATION NOTE: ATS MEMORY ATTRIBUTES §

ATS Memory Attributes (AMAs) are used with bus transactions on the Internal Coherent Bus (ICB) of devices as shown in the figure below. AMAs simplify processing at the TA. AMAs are simply passed through to the ICB avoiding a look-up at the TA.

AMAs may be supplied by a Translation Completion and stored in the ATC. The Endpoint may provide AMAs in Memory Read/Write TLPs using a TPH TLP Prefix. AMAs may also be provided using Vendor-Defined Messages, however such methods are outside the scope of this specification.

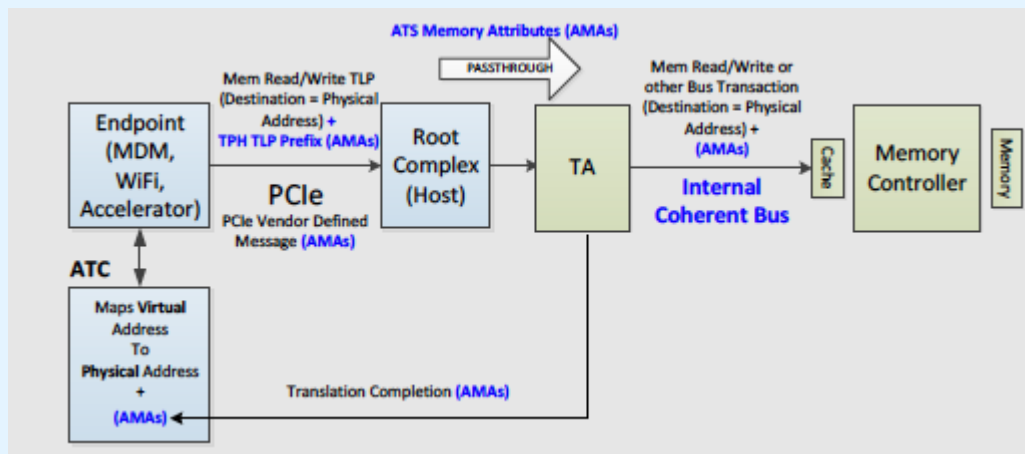


Figure 10-17 ATS Memory Attributes Example §

AMAs provide hints on performing cache management as well as behavior on the ICB for a particular transaction. For example, AMAs may hint:

- Data is cacheable or non-cacheable.
- Write-hit policy. “Write back” may indicate that on a cache hit, update the cache, do not update memory. Mark the cache as dirty.
- Write-miss policy. “Write allocate” may indicate that on a cache miss, update memory, bring the data block to the cache (allocate).

AMAs are similar to TLP Processing Hints but should be considered independent. Hence all permutations of AMAs and TLP Processing Hints are permitted. However, AMAs may convey different semantics than what is provided by TLP Processing Hints. The cache behavior conveyed by AMAs and TLP Processing Hints may not align. Additionally, AMAs may convey semantics not addressed by TLP Processing Hints. For example, handling of transactions on the ICB when caching is not in use, such as, “reordering permitted/not permitted”, “early write acknowledgement permitted/not permitted.”

AMAs may be transmitted in peer-to-peer transactions. It is recommended that an application programming interface (API) be provided to enable AMAs to be configured into the Translation Agent (TA). Note: the TA might not be collocated with the Completer.

An example of memory attributes such as “strongly ordered device normal” may be found at <http://infocenter.arm.com>.

10.2.3.10 TEE Exclusive Memory Attribute (TE) §

Trusted Execution Environments (TEEs) are associated with two types of memory: TEE memory and non-TEE memory. TEE memory includes the memory used for the TEE's code, data, and execution state. Additionally, the memory in TEE Device Interfaces (TDIs) that are associated with the TEE and have the IS_NON_TEE_MEM attribute Cleared also falls under the TEE memory. All other memory related to the TEE is classified as non-TEE memory. TEE memory is protected for confidentiality and integrity by restricting access only to entities within the TEE's Trusted Computing Base (TCB). Conversely, non-TEE memory is accessible to entities outside the TEE's TCB, and its contents are not considered trustworthy by the TEE.

The TEE Exclusive Memory Attribute (TE) bit, when Set indicates that the Translated Addresses correspond to locations in TEE memory. When the TEE Exclusive Memory Attribute (TE) bit is Cleared, it signifies that the Translated Addresses correspond to locations in non-TEE memory.

The rules for the use of the TEE Exclusive Memory Attribute (TE) bit by TDIs are specified in [§ Section 11.4.10](#).

Functions that support use of this bit must have the TEE Exclusive Memory Attribute Supported bit Set in the ATS Capability Register (see [§ Section 10.5.1.2](#)).

10.2.4 Completions with Multiple Translations §

An ATC is allowed to request that the TA provide translations for a virtually contiguous range of addresses. It does this by setting the Length field in the Translation Request to a value that is two times the number of requested translations as long as the request size (Total Completion Length * 4) is not larger than Read Completion Boundary (RCB) in the [Link Control Register](#).

If multiple translations are requested, the TA may return one or more translations as long as the number of translations does not exceed the number of requested translations. It is not an error for the TA to return fewer translations than requested and no error indication is sent unless there is an error in accessing the data.

If the Translation Completion contains multiple translations, all translations must have the same indicated size. Also, successive translations must apply to the virtual address range that abuts the previous translation in the same completion.

If a translation has both R = 0b and W = 0b, the TA must still set the Size field and the lower bits of the Translated Address field used to encode the completion size to appropriate values.

Each translation in a Translation Completion will have some overlap with the implied memory range of the Translation Request (see [§ Section 10.2.2](#)).

A successful Translation Completion must consist of one or two CplDs. Each CplD must contain an integral number of Translations (i.e., Length must be a multiple of 2).

The TA is permitted to choose:

- the number of translations it returns for each Translation Request (e.g., Byte Count of the first or only CplD)
- if it returns more than one translation, whether it uses one or two CplDs
- if it returns two CplDs, how many translations are returned in each CplD

The Byte Count and Length fields in each CplD is used to convey these choices to the ATC. The Lower Address field should not be needed by the ATC (its value is computed as defined in [§ Section 10.2.3](#) to satisfy RCB rules but the field otherwise conveys no additional information).

- If a Translation Completion CplD has a Byte Count that is greater than four times the Length field, then this is the first of two CplDs for the transaction.
- If a Translation Completion CplD has a Byte Count that is equal to four times the Length field, then this is the second or only CplD for the request.

Note: There are multiple reasons that the TA may truncate the results of the completion. For example, the request might ask for a range of addresses, not all of which are defined. This could occur if the first translation is valid but located at the end of a page of translations. The TA, in looking up the next page of translations, may find that the page is not valid so the addresses are not valid. The range of addresses that are valid would be returned and no error indicated. When truncating a Translation Completion the TA is not allowed to pad the response with invalid entries (R = 0b, W = 0b).

Note: There are multiple reasons that the TA may break a Translation Completion into multiple TLPs. As an example, if the virtual address of the Translation Completion resolves to a table access that crosses an implementation specific address boundary, the completion to the TA may be broken into two completions. Rather than require that the TA accumulate the results, the TA is permitted to send each portion of the Translation Completion to a Function when it is received from memory.

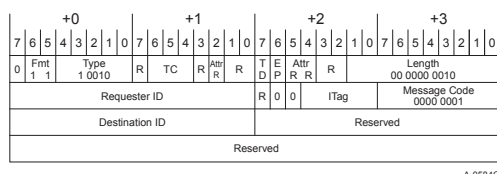
10.3 ATS Invalidation §

ATS uses the messages shown in this section to maintain consistency between the TA and the ATC. This specification assumes there is a single TA associated with each ATC. The TA (in conjunction with its associated software) must ensure that the address translations cached in the ATC are not stale by issuing Invalidate Requests.

10.3.1 Invalidate Request §

When a translation is changed in the TA and that translation might be contained within an ATC in a Function, the TA (in conjunction with its associated software) must send an Invalidate Request to the ATC to maintain proper synchronization between the ATPT and the ATC. An Invalidate Request is used to clear a specific subset of the address range from the ATC. Invalidate Requests are constrained to cover power of 2 multiple of 4096-byte pages.

The format of an Invalidate Request is shown in § Figure 10-18 and § Figure 10-19.



A-0584C

Figure 10-18 Invalidate Request Message - Non-Flit Mode §

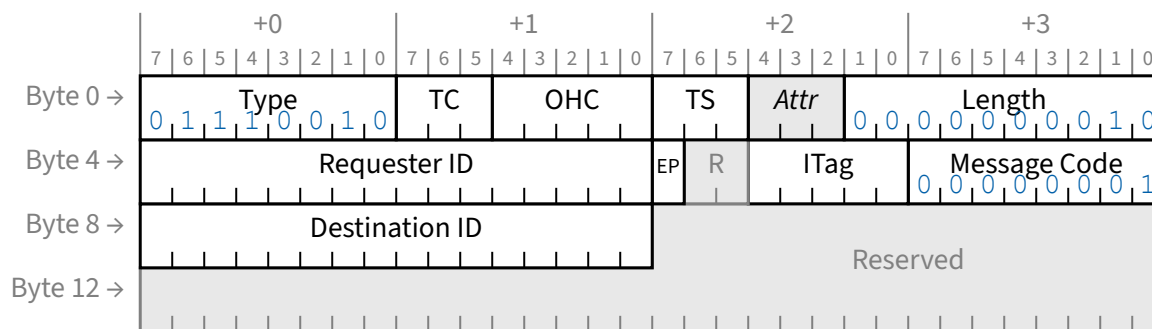


Figure 10-19 Invalidate Request Message - Flit Mode §

The Invalidate Request is a MsgD transaction with 64 bits of data. Invalidate Request messages may be sent in any TC. Invalidate Request Messages optionally include a PASID (see § Section 10.3.8)

- In Non-Flit Mode, PASID is included when a PASID TLP Prefix is present. The Execute Requested and Privileged Mode Requested bits in this PASID TLP Prefix are Reserved.
- In Flit Mode, PASID is included when OHC-A4 is present and the PV is Set. PASID is not included if either OHC-A4 is not present or if PV is Clear. OHC-A4 also contains Destination Segment if DSV is Set ²¹⁹.

The ITag field is used by the TA to uniquely identify Invalidate Requests it issues. A TA must ensure that once an ITag is used, it is not reused for the same Destination ID until either released by the corresponding Invalidate Completions or by a vendor-specific timeout mechanism (see below). A TA is permitted to use the same ITag value for Invalidate Requests that target different Destination IDs. See the Invalidate Queue Depth field in the ATS Extended Capability for additional considerations.

The address range specified in an Invalidate Request may span one or more STU 4096-byte pages. Invalidation ranges are required to be naturally aligned and should not be smaller than STU 4096-byte pages. Upon receiving an Invalidate Request with a range less than STU an ATC may either (1) signal an Unsupported Request (not recommended) or (2) round the range of the request up to a value greater than or equal to the STU.

IMPLEMENTATION NOTE: INVALIDATE COMPLETION TIMEOUT §

Devices should respond to Invalidate Requests within 1 minute (+50% -0%). Having a bounded time permits an ATPT to implement Invalidate Completion Timeouts and reuse the associated ITag values. ATPT designs are implementation specific. As such, Invalidate Completion Timeouts and their associated error handling are outside the scope of this specification.

The content of the payload is the untranslated address range to be invalidated. The payload format is shown in § Figure 10-20.

219. Invalidate Request Messages are issued by the TA through a specific RP. Since everything below a given RP is part of the same segment, the Destination Segment field in the message is not required unless needed by the RC to determine the appropriate RP.

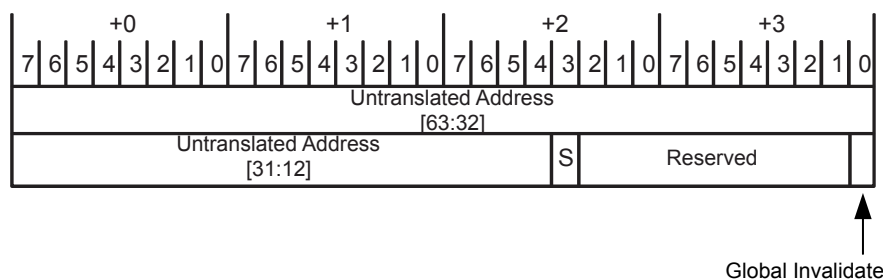


Figure 10-20 Invalidate Request Message Body §

The S field is used to indicate if the range being invalidated is greater than 4096 bytes. Its meaning is the same as for the Translation Completion (see § Section 10.2.3.1 and § Section 10.2.3.2).

The Global Invalidate bit indicates that the Invalidate Request Message affects all PASID values (see § Section 10.3.8). This bit is Reserved unless the Invalidate Request has a PASID. The bit is ignored by the ATC if Global Invalidate Supported bit is Clear (see § Section 10.3.8).

IMPLEMENTATION NOTE: INVALIDATE REQUESTS AND FUNCTION LEVEL RESET & DEVICE POWER STATE TRANSITIONS §

Invalidate Requests received while a Function is undergoing Function Level Reset, or is in (or transitioning to) non-D0 device state, may be dropped by the Function. Similarly, Invalidate Requests already received but pending at the time of receiving initiate FLR or D-state transition request may be dropped the Function.

System software can avoid ATS invalidation race conditions on Function Level Reset and Device Power State transitions in a variety of ways. For example:

1. When disabling ATS on a Function, system software quiesces ATS invalidations to the Function (i.e., either responses are received for all Invalidate Requests issued to the Function, or any pending Invalidate Requests to the Function have timed out).
2. Software ensures no Invalidate Requests are issued to a Function when its ATS capability is disabled.
3. Before initiating the FLR (or Device power state transitions), software disables ATS as described in item 1. above.

10.3.2 Invalidate Completion §

When a Function completes an invalidate operation, it will send one or more Invalidate Completion messages to the TA. These messages must be tagged with information extracted from the Invalidate Request to enable the TA to associate the Invalidate Completions with the Invalidate Request.

The format of the Invalidate Completion message is shown in § Figure 10-21 and § Figure 10-22.

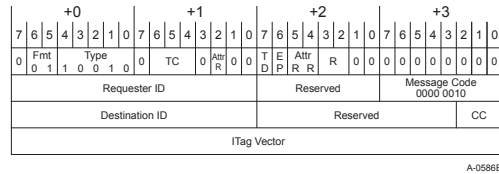


Figure 10-21 Invalidate Completion Message Format - Non-Flit Mode §

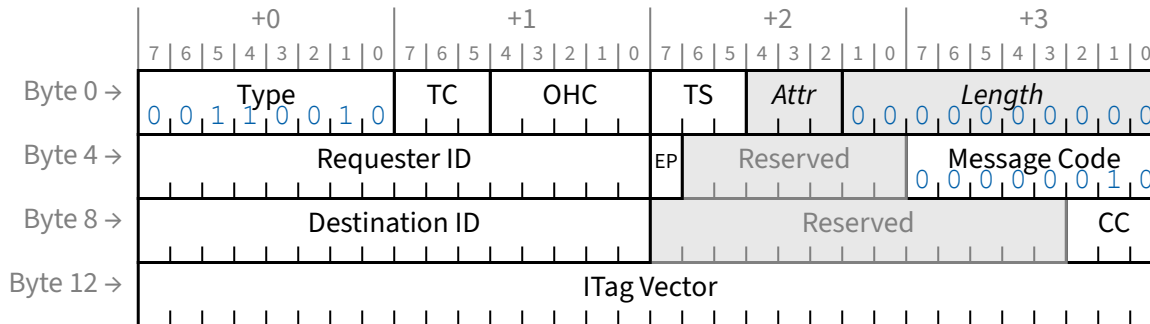


Figure 10-22 Invalidate Completion Message - Flit Mode §

The Invalidate Completion message is a Msg transaction routed by ID. The Requester ID field of the Invalidate Completion message is set to the Requester ID of the Function containing the ATC. The Destination ID field of the Invalidate Completion is set to the Requester ID of the TA. The ATC may derive the Requester ID of the TA from the Requester ID field of the corresponding Invalidate Request. Alternatively, since the ATC is only associated with a single TA, the ATC may sample and store the Requester ID from the first Invalidate Request following a Fundamental Reset or FLR. Subsequent Invalidate Completion messages may use this value to set the Destination ID field of Invalidate Completion messages.

In Flit Mode, Invalidate Completion messages optionally contain OHC-A4 when a Destination Segment is needed (e.g., the TA and the ATC are in different segments). The PASID is Reserved. PV must be 0b.

The Completion Count (CC) field indicates the number of individual Invalidate Completion messages that must be sent for the associated Invalidate Request. Setting the CC field to 0 indicates that eight responses must be sent. The TA is responsible for collecting all the responses associated with a given Tag before considering the corresponding Invalidate Request to be complete.

Invalidate Completion messages may be sent on any TC, independent of the TC the originating Invalidate Request was received. This enables implementations to utilize the Invalidate Completion to push outstanding transactions to the TA to guarantee the required invalidation semantics are met. Implementations that utilize a single Upstream TC are required to send a single Invalidate Completion in the utilized TC.

The ITag Vector field is used to indicate which Invalidate Request has been completed. Each of the 32 possible ITag field values from the Invalidate Request is represented by a single bit in the ITag Vector field. The least significant bit (bit 0; i.e., the right-most bit in the schematic representation of the Invalidate Completion message shown in § Figure 10-21 and § Figure 10-22) of the ITag Vector field corresponds to the ITag field value of 0. The most significant bit (bit 31) of the ITag Vector field corresponds to the ITag field value of 31. Implementations are allowed to coalesce multiple Invalidate Completions by setting multiple ITag Vector bits in a single message provided the following conditions are met:

- The Invalidate Completions flow in the same TC.
- The Invalidate Completions have the same CC value.
- All fragments of an Invalidate Completion must have identical Request ID, CC, and ITag Vector fields.

A TA that receives an Invalidate Completion for an ITag that has no outstanding Invalidate Request shall report this error using implementation specific mechanisms. One possible such mechanism is to report the Invalidate Completion as an Unexpected Completion (UC).

Functions that do not support ATS will treat an Invalidate Request as UR.

Functions supporting ATS are required to send an Invalidate Completion in response to a Invalidate Request independent of whether the Bus Master Enable bit is Set or not. Note that the above conditions must be satisfied even when Bus Master Enable is Cleared. The method for a device to achieve this is implementation dependent.

IMPLEMENTATION NOTE: BUS MASTER ENABLE CHANGE §

When Bus Master Enable changes from Set to Clear, no further memory requests should be queued. It is possible that queued write requests are present when BME is Cleared. These requests could block an Invalidate Completion. These requests must be either sent or dropped. This will ensure that all outstanding write transactions that are potentially dependent upon the outstanding invalidation are complete.

10.3.3 Invalidate Completion Semantics §

Before an ATC can return an Invalidate Completion for a given Invalidate Request, it must ensure the following conditions are satisfied with respect to stale addresses:

- All new Translated Memory Requests initiated by the Function (or its Shadows, or its Scalable Device Interfaces) will not utilize stale address translations.
- All expected Completions for Translated Memory Requests initiated by the Function (or its Shadows, or its Scalable Device Interfaces) and referencing stale addresses must be received.
- All expected Translation Completions for Translation Requests initiated by the Function (or its Shadows, or its Scalable Device Interfaces) and referencing stale addresses must be received or tagged to be discarded.
- All outstanding non-UIO Posted Translated Memory Requests initiated by the Function (or its Shadows, or its Scalable Device Interfaces) and referencing stale addresses must be ensured to be transmitted ahead of the associated Invalidate Completion(s).

The ATC is required to send a copy of the Invalidate Completion message in each TC in which a non-UIO Posted Translated Memory Request has been issued but not known to have been pushed to observability on the fabric through the TA. The CC field must be set to the same value in each copy of the Invalidate Completion message indicating number of copies sent. The TA is responsible for collecting all sent responses before considering the invalidation to be complete. The ATC must send at least one Invalidate Completion message.

The ATC must not send copies of the Invalidate Completion message in UIO VCs (Posted Message TLPs are not defined for UIO VCs). When all traffic affected by an ATS Invalidate Request is UIO, it is implementation specific how the ATC picks a non-UIO TC to use for the Invalidate Completion message.

A TA must ensure forward progress of Requests without dependence on receiving Invalidate Completions.

Invalidate Completion Messages must contain the Requester ID used in the associated Invalidate Request.

When Shadow Functions are enabled, it is sufficient to issue a single Invalidate Request message targeting either the Function or one of its Shadows. It is recommended that system software issue an Invalidate Request Message to the “main” Function instead of a Shadow Function. Since IDO is Clear, the Invalidate Completion resulting from that Invalidate Request will push to the TA any earlier Posted Writes for both the Function and its Shadows.

Invalidate Completion behavior is independent of whether the associated Invalidate Request was issued to the Function or one of its Shadows.

IMPLEMENTATION NOTE:

IMPLIED TC FLUSHING §

When making the decision as to which TC to send Invalidate Completions, an ATC may infer, in an implementation specific manner, that an issued posted write has been pushed to the TA. For example, a Function that has sent a read transaction to a destination above the TA and received its corresponding response may infer that any preceding posted writes issued in the same TC have been pushed to the TA.

Independent of this optimization, the ATC is required to send at least one Invalidate Completion.

10.3.4 Request Acceptance Rules §

In accord with the request acceptance rules enumerated in this section, a Function is not allowed to create a dependency in which the acceptance of a posted transaction is dependent upon the transmission of a posted transaction. Given Invalidate Requests and Invalidate Completions both are posted transactions, Functions must not make the acceptance of an Invalidate Request dependent upon the transmission of an Invalidate Completion. The method for achieving this is implementation specific.

A Function with an ATS capability in its configuration space must be able to accept Invalidate Requests and send Invalidate Completions even if ATS is not enabled.

IMPLEMENTATION NOTE:

INVALIDATE QUEUE DEPTH §

An ATC is only associated with a single TA. Each TA is limited to a total of 32 outstanding invalidations to any given ATC. This limits the number of outstanding Invalidate Requests active to a single ATC to 32. To avoid a post-to-post dependency, an ATC is required to accept up to 32 Invalidate Requests.

An ATC may choose to implement a maximally sized input queue holding Invalidate Requests. Alternatively, an ATC may choose to implement a maximally sized output queue holding Invalidate Completions. Note that queuing Invalidate Completions requires significantly less state per entry resulting in a potentially more efficient implementation than input queue buffering.

Note that the choice of whether to implement input queuing or output queuing (or a hybrid of both) has no impact on ensuring deadlock free behavior. But implementation choices with regard to queuing may have a significant impact on performance (see § Section 10.3.5).

10.3.5 Invalidate Flow Control §

Due to the variety of caching architectures and queuing strategies, implementations may vary greatly with respect to invalidation latency and throughput. It is possible that a TA may generate Invalidate Requests at a rate that exceeds the average ATC service rate. When this happens, the credit based flow control mechanisms will throttle the TA issue rate. A side effect of this is congestion spreading to other channels and Links through the credit based flow control mechanism. Depending on the frequency and duration of this congestion, performance may suffer. It is strongly recommended that TA and its associated software implement higher level flow control mechanisms.

To assist with the implementation of Invalidate Flow Control, an ATC must publish the number of Invalidate Requests it can buffer before back pressuring the Link. This field applies to all invalidations serviced by the Function, independent of the size of the invalidation. This value is communicated in the Invalidate Queue Depth field in the ATS capability structure (see § Section 7.8.9). A value of 0 0000b indicates that invalidate flow control is not necessary to this Function.

IMPLEMENTATION NOTE: INVALIDATE FLOW CONTROL §

A Function may indicate that invalidate flow control is not required when one or more of the following is true:

1. The Function can handle invalidations at the maximum arrival rate of Invalidate Requests.
2. The Function will not or very rarely cause Link backpressure (performance loss is negligible).
3. The Function can fully buffer the maximum number of incoming invalidations without back pressuring the Link.

Each Function is required to implement sufficient queuing to ensure it can hold the maximum number of outstanding Invalidate Requests from a TA (using either input or output queuing). However, with a PF containing either an SR-IOV Extended Capability or an SIOV Extended Capability shares a single Invalidate Request queue in the PF with all of its VFs or SDIs. To implement invalidation flow control, the TA must ensure that the total number of outstanding Invalidate Requests targeting the shared PF queue does not exceed the value in the PF's Invalidate Queue Depth field.

10.3.6 Invalidate Ordering Semantics §

Invalidate Requests and Translation Completions may be sent using different TC and are, therefore, unordered with respect to each other (from the Link's perspective). An ATC must ensure that the proper invalidation behavior is maintained when an Invalidate Request bypasses a Translation Completion to an overlapping region.

An ATC must “snoop” its outstanding translation request queue against all arriving Invalidate Requests. When snooping a request for a $N \times \text{STU}$ sized translation (N is a power of 2), the ATC must snoop the range of addresses starting at the STU aligned region containing the specified address and ending $(N-1)$ STU size pages later.

If an Invalidate Request overlaps the address range in an outstanding Translation Request, the Translation Request must be tagged as invalid and the results of its corresponding Translation Response must be discarded prior to transmission of the Invalidate Completion. If the Translation Response is received before the Invalidate Completion is sent, an implementation is free to issue requests utilizing the translation result provided the Invalidate Completion Semantics (see § Section 10.3.3) are satisfied.

IMPLEMENTATION NOTE: REQUEST RANGE OVERLAP IN INVALIDATIONS §

In the description above, N is the number of STU sized translations that were requested in the Translation Request. This is equal to (Length field in Translation Request)/2.

As an example:

STU is 00 0010b indicating 16384-byte pages.

An outstanding Translation Request has a Length field of 00 0000 0100b indicating two translations covering a range of 32768 bytes.

The high-order 48 bits of the Translation Request are 0000 0FFF FFFFh.

The low-order 16 bits of the address in the request are 11xx xxxx xxxx xxxb indicating that the translation request covers a range that overlaps a 32768-byte boundary (in fact, the request crosses a 16-TB boundary).

If two translations are returned, they would cover the two STU sized regions at 0000 0FFF FFFF C000h and 0000 1000 0000 0000h.

An Invalidate Request is received with the high-order 48 bits of 0000 1000 0000h and the low-order 16 bits of 0001 1xxx xxxx xxxb.

The ATC must detect that a translation associated with a portion of the Translation Request is now invalidated and the Translation Completion associated with the invalidated region must be discarded (for simplification, the ATC is allowed to discard all of the Translation Completion).

It should be noted that, processing of the Invalidate Requests is simplified if Translation Requests do not cross alignment boundaries of the request. The Translation Request from the above example is not aligned to a 32768-byte boundary. If it were broken into two requests, it would be simpler to associate the range of the Invalidate Request with the address in the Translation Request. Breaking the Translation Requests into aligned requests is not a requirement.

10.3.7 Implicit Invalidation Events §

The following events will cause the invalidation of all ATC entries:

- Conventional Reset (all forms)
- Function Level Reset
- E field in ATS Capability changes from Clear to Set

The following events will cause the invalidation of all non-Global Mapping ATC entries that were requested using a specific PASID:

- Stopping the use of a PASID as defined in this specification.

No explicit Invalidate Completion message is sent when these implied invalidate events occur.

IMPLEMENTATION NOTE:

IMPLICIT INVALIDATION AND PASID §

Software may not change any of the PASID enable bits when the E field in the ATS Capability is Set. The invalidation that occurs when software Sets the E field also invalidates ATC entries with an associated PASID value.

10.3.8 PASID and Global Invalidate §

The requirements in this section apply to Functions that support the PASID. For Invalidate Requests that have a PASID, the ATC shall:

- Optionally signal Unsupported Request (UR) if the associated PASID value is greater than or equal to $2^{\text{Max PASID Width}}$. This error may be signaled anytime an out of range PASID value is present, even when the PASID value is ignored (see below).
- Return an Invalidate Completion if PASID Enable is Clear.
- If the Function supports Global Invalidate (see § Section 7.8.9.2):
 - If the Global Invalidate bit in the Request is Set, invalidate Global and non-Global Mapping entries in the ATC within the indicated memory range associated with any PASID value and return an Invalidate Completion. The PASID value in the PASID is ignored.
 - If the Global Invalidate bit in the Request is Clear, invalidate only non-Global Mapping entries in the ATC within the indicated memory range that were requested using the associated PASID value and return an Invalidate Completion.

Global Mapping entries in the ATC for some or all of the indicated memory range may be retained.

- If the Function does not support Global Invalidate (see § Section 7.8.9.2), invalidate entries in the ATC within the indicated memory range that were requested using the associated PASID value and return an Invalidate Completion.
- If no matching entries are present in the ATC, invalidate no ATC entries and return an Invalidate Completion.

For Invalidate Requests that do not have a PASID, the ATC shall:

- Invalidate ATC entries within the indicate memory range that were requested without a PASID value.
- Invalidate ATC entries at all addresses that were requested with any PASID value.

10.4 Page Request Services §

The general model for a page request is as follows:

1. A Function determines that it requires access to a page for which an ATS translation is not available.
2. The Function causes the associated Page Request Interface to send a Page Request Message to its RC. A Page Request Message contains a page address and a Page Request Group (PRG) index. The PRG index is used to identify the transaction and is used to match requests with responses.

3. When the RC determines its response to the request (which will typically be to make the requested page resident), it sends a PRG Response Message back to the requesting Function.
4. The Function can then employ ATS to request a translation for the requested page(s).

A Page Request Message is a PCIe Message Request that is Routed to Root Complex with a Message Code of 4 (0000 0100b). The mechanism employed at the RC to buffer requests is implementation specific. The only requirement is that an RC not silently discard requests.

All Page Request Messages and PRG Response Messages travel in PCIe Traffic Class 0. A Page Request Message or PRG Response Message with a Traffic Class other than 0 shall be treated as Malformed TLPs by the RC or endpoint that receives the same. Intermediate routing elements (e.g., Switches) shall not detect this error.

The Relaxed Ordering and ID-Based Ordering bits in the Attr field of Page Request Messages and PRG Response messages may be used. The No Snoop bit in the Attr field is reserved.

The page request service allows grouping of page requests into Page Request Groups (PRGs). A PRG can contain one or more page requests. All pages in a PRG are responded to en masse by the host. Individual pages within a PRG are requested with independent Page Request Messages and are recognized as belonging to a common PRG by sharing the same PRG index. The last request of a PRG is marked as such within its Page Request Message. One request credit is consumed per page request (not per PRG).

A PRG Response Message is a PCIe Message Request that is Routed by ID back to the requesting Function. It is used by system software to alert a Function that the page request(s) associated with the corresponding PRG has (have) been satisfied. The page request mechanism does not guarantee any request completion order and all requests are inherently independent of all other concurrently outstanding requests. If a Function requires that a particular request be completed before another request, the initial request will need to complete before the subsequent request is issued. It is valid for a Function to speculatively request a page without ascertaining its residence state and/or to issue multiple concurrently outstanding requests for the same page.

A Page Request Interface is allocated a specific number of page request message credits. An RC (system software) can divide the available credits in any manner deemed appropriate. Any measures the host chooses to employ to ensure that credits are correctly metered by Page Request Interfaces (a Page Request Interface is not using more than its allocation) is an implementation choice. A Page Request Interface is not allowed to oversubscribe the available number of requests (doing so can result in the page request mechanism being disabled if the buffer limit is exceeded at the root). A Page Request Interface's page request allocation is static. It is determined when the Page Request Interface is enabled and can only be changed by disabling and then re-enabling the interface.

10.4.1 Page Request Message §

A Function uses a Page Request Message to send page requests to its associated host. A page request indicates a page needed by the Function. The Page Request Interface associated with a Function is given a specific Page Request allocation. A Page Request Interface shall not issue page requests that exceed its page request allocation.

A page request contains the untranslated address of the page that is needed, the access permissions needed for that page, and a PRG index. A PRG Index is a 9-bit scalar that is assigned by the Function to identify the associated page request. Multiple pages may be requested using a single PRG index. When more than a single page is to be associated with a given PRG, the Last flag in the Page Request Record is cleared in all the requests except the last request associated with a given PRG (the flag is set in the last request). Page requests are responded to en masse. No response is possible (except for a Response Failure error) until the last request of a PRG has been received by the root. The number of PRGs that a Function can have outstanding at any given time is less than or equal to the associated Page Request Interface's Outstanding Page Request Allocation. It is valid for a request group to contain multiple requests for the same page and for multiple outstanding PRGs to request the same page.

A Page Request Interface applies to the "main" Function and its enabled Shadow Functions (where the "main" is the Function that contains both the Page Request Extended Capability and the Shadow Function Extended Capability). All

Page Request Messages of a single PRG must have the same Requester ID. When Shadow Functions are enabled, it is recommended that Page Request Messages use the Requester ID of the “main” Function

The first two DWs of a Page Request Message contain a standard PCIe message header. The second two DWs of the message contain page request specific data fields.

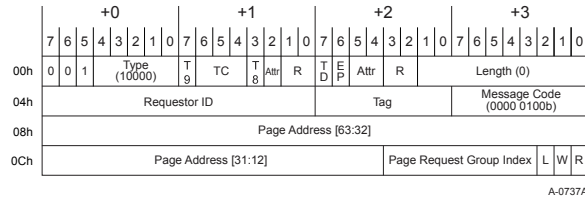


Figure 10-23 Page Request Message - Non-Flit Mode §

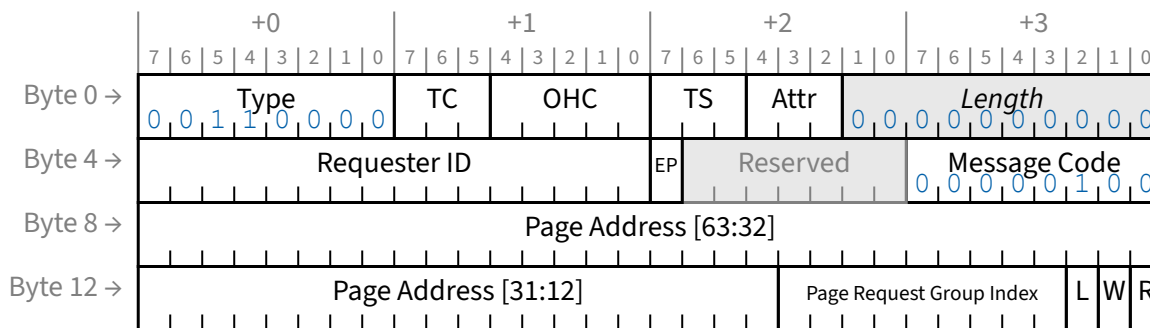


Figure 10-24 Page Request Message - Flit Mode §

Table 10-6 Page Request Message Data Fields §

Field	Meaning
R	Read Access Requested - This field, when Set, indicates that the requesting Function seeks read access to the associated page. When Clear, this field indicates that the requesting Function will not read the associated page. The R field must be Set for Page Requests with a PASID and that have the <u>Execute Requested</u> bit Set. If R and W are both Clear and L is Set, this is a Stop Marker (see § Section 10.4.1.2.1).
W	Write Access Requested - This field, when Set, indicates that the requesting Function seeks write access and/or zero-length read access to the associated page. When Clear, this field indicates that the requesting Function will not write to the associated page. Upon receiving a Page Request Message with the W field Set, the host is permitted to mark the associated page dirty. Thus, Functions must not issue such Requests unless the Function has been given explicit write permission. If R and W are both Clear and L is Set, this is a Stop Marker (see § Section 10.4.1.2.1).
L	Last Request in PRG - This field, when Set, indicates that the associated page request is the last request of the associated PRG. A PRG can have a single entry, in which case the PRG consists of a single request in which this field is Set. When Clear, this field indicates that additional page requests will be posted using this record's PRG Index.

Field	Meaning
	If R and W are both Clear and L is Set, this is a Stop Marker (see § Section 10.4.1.2.1).
Page Request Group Index	Page Request Group Index - This field contains a Function supplied identifier for the associated page request. A Function need not employ the entire available range of PRG index values. A host shall never respond with a PRG Index that has not been previously issued by the Function and that is not currently an outstanding request PRG Index (except when issuing a Response Failure, in which case the host need not preserve the associated request's PRG Index value in the error response).
Page Address	Page Address - This field contains the untranslated address of the page to be loaded. For pages larger than 4096 bytes, the least significant bits of this field are ignored. For example, the least significant bit of this field is ignored when an 8096-byte page is being requested.

IMPLEMENTATION NOTE: LAST BIT AND RELAXED ORDERING §

If multiple page requests are associated with a single PRG index, the last page request of a PRG should have the Relaxed Ordering attribute bit Clear in addition to having the Last flag Set. All other page request messages may have the Relaxed Ordering attribute bit set to any value.

10.4.1.1 PASID Usage §

The PASID Extended Capability indicates whether a Function supports PASID TLP Prefixes (NFM) or OHC-A1 with PASID (FM), and whether it is enabled to send and receive them.

Functions that support PASID are permitted to send a PASID on Page Request Messages. The PASID field contains the process address space of the page being requested and the Execute Requested and Privileged Mode Requested bits indicate the access being requested. In FM, the NW bit and the Last DW BE / 1st DW BE fields in the OHC-A1 are Reserved.

If one Page Request Message in a PRG has a PASID, all Page Request Messages in that PRG must contain identical PASID values. Behavior is undefined when the PASID values are inconsistent.

Functions that support PASID and have the PRG Response PASID Required bit Set (see § Section 10.5.2.3), expect that PRG Response Messages will contain a PASID if the associated Page Request Message had a PASID. For such PRG Response Messages, the Execute Requested and Privileged Mode Requested bits are reserved and the PASID field contains the PASID from the associated Page Request Message.

10.4.1.2 Managing PASID Usage on PRG Requests §

There are rules for stopping and starting the use of a PASID.

This section describes additional rules that apply to Functions that have issued Page Request Messages in a PASID that is being stopped. No additional rules are required to start the usage of the Page Request Interface for a PASID.

When stopping the use of a particular PASID, a Stop Marker Message may be optionally used to avoid waiting for PRG Response Messages before the Function indicates that the stop request for a particular PASID has completed.

To stop without using a Stop Marker Message, the Function shall:

1. Stop queueing new Page Request Messages for this PASID.
2. Finish transmitting any multi-page Page Request Messages for this PASID (i.e., send the Page Request Message with the L bit Set).
3. Wait for PRG Response Messages associated any outstanding Page Request Messages for the PASID.
4. Indicate that the PASID has stopped using a device specific mechanism. This mechanism must indicate that a Stop Marker Message will not be generated.

To stop with the use of a Stop Marker Message the Function shall:

1. Stop queueing new Page Request Messages for this PASID.
2. Finish transmitting any multi-page Page Request Messages for this PASID (i.e., send the Page Request Message with the L bit Set).
3. Internally mark all outstanding Page Request Messages for this PASID as stale. PRG Response Messages associated with these requests will return Page Request Allocation credits and PRG Index values but are otherwise ignored.²²⁰
4. Indicate that the PASID has stopped using a device specific mechanism. This mechanism must indicate that a Stop Marker Message will be generated.
5. Send a Stop Marker Message to indicate to the host that all subsequent Page Request Messages for this PASID are for a new use of the PASID value.

Note: Steps 4 and 5 may be performed in either order, or in parallel.

10.4.1.2.1 Stop Marker Messages §

A Stop Marker Message indicates that a Function has stopped using the Page Request Interface and has transmitted all pending Page Request Messages for a specific PASID. Stop Marker Messages are strongly ordered with respect to Page Request Messages and serve to push Page Request Messages toward the Host. When the Host receives the Stop Marker Message, this indicates that all Page Request Messages associated with the PASID being stopped have been delivered and that any subsequent Page Request Message with the same PASID value are associated with a new incarnation of that PASID value.

Stop Marker Messages do not have a response. They do not have a PRG Index and do not consume Page Request allocation (see § Section 10.5.2.5).

The Stop Marker Message bit layout is shown in § Figure 10-25.

220. Page Request Allocation is shared across all PASIDs of the Function (see § Section 10.5.2.5). If PRG Response PASID Required is Clear, PRG Index values are shared across all PASIDs of the Function (see § Section 10.5.2.3).

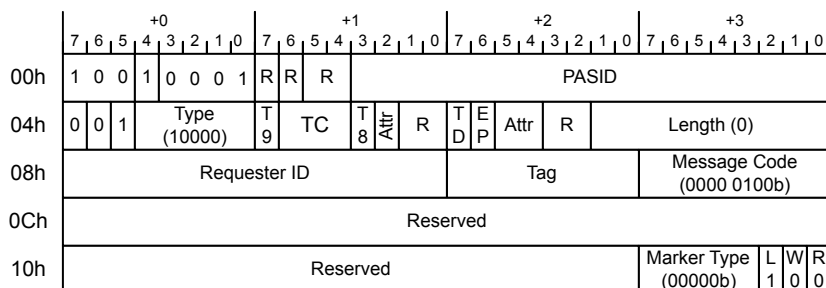


Figure 10-25 Stop Marker Message - Non-Flit Mode §

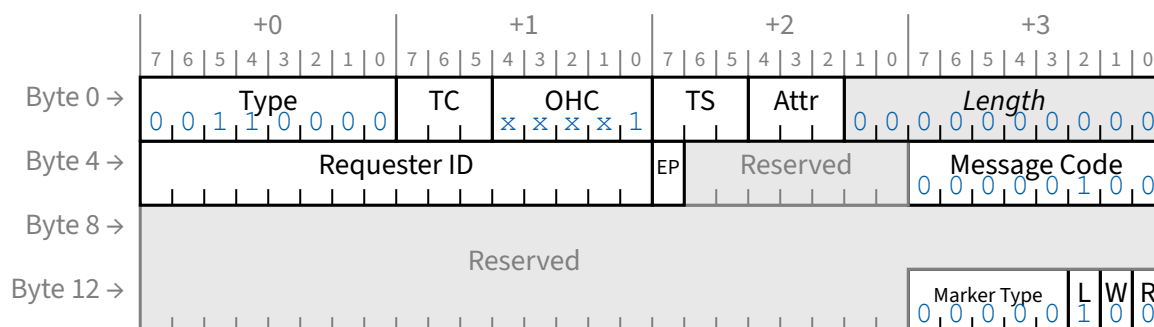


Figure 10-26 Stop Marker Message - Flit Mode §

A Stop Marker Message is encoded as a Page Request Message for which:

- In NFM, includes a PASID TLP Prefix. The Execute Requested and Privileged Mode Requested bits are Reserved.
- In FM, includes OHC-A1 with PASID. The Execute Requested bit, Privileged Mode Requested bit, and Last DW BE / 1st DW BE fields in the OHC-A1 are Reserved.
- The L, W and R fields contain 1b, 0b and 0b respectively.
- The Untranslated Address field and upper bits of the PRG Index field are Reserved.
- The Marker Type field contains 0 0000b to indicate that this is a Stop Marker Message.
- The Traffic Class must be 0.
- The Relaxed Ordering attribute bit must be Clear.
- The ID-Based Ordering attribute bit may be Set.

Behavior is undefined if a Stop Marker Message is received and any of the following are true:

- Marker Type not equal to 0 0000b.
- No PASID TLP Prefix is present (NFM).
- The PASID value does not match an outstanding stop request.
- An incomplete Page Request Message for the PASID is outstanding (i.e., for some PRG Index, the most recently received Page Request Message did not have the L bit Set).

10.4.2 Page Request Group Response Message §

System hardware and/or software communicate with a Function's page request interface via PRG Response Messages. A PRG Response Message is used by a host to signal the completion of a PRG, or the catastrophic failure of the interface. A single PRG Response Message is issued in response to a PRG, independent of the number of page requests associated with the PRG. There is no mechanism for indicating a partial request completion or partial request failure. If any of the pages associated with a given PRG cannot be satisfied, then the request is considered to have failed and the reason for the failure is supplied in the PRG Response Message. The host has no obligation to partially satisfy a multi-page request. If one of the requested pages cannot be made resident, then the entire request can, but need not, be discarded. That is, the residence of pages that share a PRG with a failed page request, but that are not associated with the failure, is indeterminate from the Function's perspective.

There are four possible Page Request failures:

1. The requested page is not a valid Untranslated Address.
2. PASID support exists, the Page Request has a PASID, and either PASID usage is not enabled for this request, the PASID value is not valid, or the Execute Requested bit is Set when R is Clear.²²¹
3. The requested page does not have the requested access attributes (including Execute permission and/or Privileged Mode access when those bits are present).
4. The system is, for an unspecified reason, unable to respond to the request. This response is terminal (the host may no longer respond to any page requests and may not supply any further replies to the Function until the Function's page request interface has been reset). For example, a request that violates a Function's assigned request limit or overflows the RC's buffering capability may cause this type of failure.

A Function's response to Page Request failure cases 1, 2, and 3 above is implementation dependent. The failure is not necessarily persistent, that is, a failed request may, in some instances succeed if re-issued. The range of possibilities precludes the precise specification of a generalized failure behavior, though on a per Function basis, the response to a failure will be an implementation dependent behavior.

All responses are sent to their associated Functions via PRG Response Messages. A Function must be capable of sinking multiple consecutive messages without losing any information. To avoid deadlock, a Function must be able to process PRG Response Messages for all of the Function's outstanding Page Request Messages without depending on the Function sending or receiving any other TLP.²²² A PRG Response Message is an ID routed PCIe message. The only Page Request Interface specific fields in this message are the Response Code and PRG. All other fields are standard PCIe message fields. (Note: these messages are routed based on the ID in bytes 8 and 9; with bytes 4 and 5 containing the host's Requester ID.)

Receipt of a PRG Response Message that contains a PRG Index that is not currently outstanding at a Function shall result in the UPRGI flag in the Page Request Extended Capability being Set, contingent upon the TLP otherwise being error free. Because of ambiguous language in earlier versions of this specification, it is permitted (though discouraged) to handle this case as an Unsupported Request (UR) or Unexpected Completion (UC) by the Function containing the Page Request Extended Capability, but otherwise no other error is permitted to be logged or signaled.

In order to prevent overflow, it is recommended to size Page Request queuing appropriately so that it does not overflow under expected behavior. If an overflow condition occurs, it is permitted to recover and resynchronize Page Request accounting as follows:

- The TA, upon detecting the overflow condition, stops accepting all incoming Page Requests for the queue and generates Page Request Group Responses with a Response Code of Success to Page Requests with L=1.

221. Behavior when PASID support does not exist is defined in § Section 6.20.1.

222. For example, processing a PRG Response Message that causes the Function to send a TLP Upstream must not block processing of subsequent downstream TLPs even if the Upstream TLP is delayed by flow control.

- Software processes all entries in the queue and generates Page Request Group Responses with a Response Code of Success to all Page Requests with L=1.
- Software clears the overflow condition to re-enable Page Request acceptance for the queue in the TA after all entries in the queue have been processed.

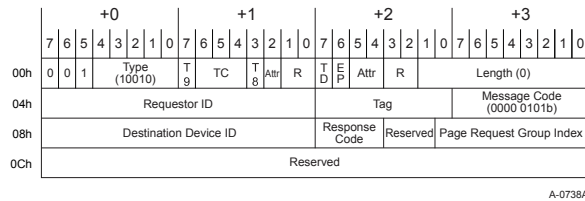


Figure 10-27 PRG Response Message - Non-Flit Mode §

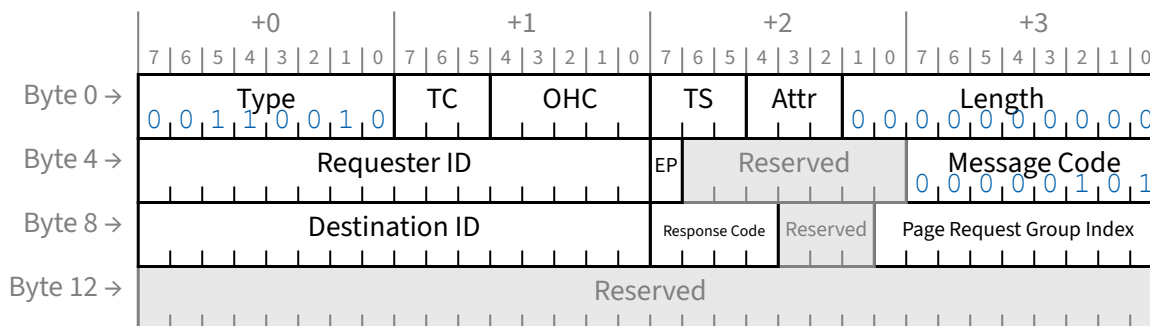


Figure 10-28 PRG Response Message - Flit Mode §

Table 10-7 PRG Response Message Data Fields §

Field	Meaning
Page Request Group Index	Page Request Group Index - This field contains a Function supplied index to which the RC is responding. A given PRG Index will receive exactly one response per instance of PRG (with the possible exception of a Response Failure).
Response Code	Response Code - This field contains the response type of the associated PRG. The encodings are presented in § Section 10.4.2.1.

10.4.2.1 Response Code Field §

The values and meaning for the Response Code field are listed in § Table 10-8.

Table 10-8 Response Codes §

Value	Status	Meaning
0000b	Success	All pages within the associated PRG were successfully made resident.
0001b	Invalid Request	One or more pages within the associated PRG do not exist or requests access privilege(s) that cannot be granted. Unless the page mapping associated with the Function is altered, re-issuance of the associated request will never result in success.

Value	Status	Meaning
1110b:0010b	Unused	Unused Response Code values. A Function receiving such a message shall process it as if the message contained a Response Code of Response Failure.
1111b	Response Failure	One or more pages within the associated request group have encountered/caused a catastrophic error. This response disables the Page Request Interface at the Function. Any pending page requests for other PRGs will be satisfied at the convenience of the host. The Function shall ignore any subsequent PRG Response Messages, pending re-enablement of the Page Request Interface.

10.4.2.2 PASID Usage on PRG Responses §

If a Page Request has a PASID, the corresponding PRG Response Message may optionally contain one as well.

If the PRG Response PASID Required bit is Clear, PRG Response Messages do not have a PASID.

If the PRG Response PASID Required bit is Set, PRG Response Messages have a PASID if the Page Request also had one. The Function is permitted to use the PASID value from the prefix in conjunction with the PRG Index to match requests and responses.

When a PASID is attached to PRG Response Messages, the Execute Requested and Privileged Mode Requested bits are Reserved and the PASID value is copied from the PASID value of the Page Request.

10.5 ATS Configuration §

10.5.1 ATS Extended Capability §

Each Function that supports ATS (capable of generating Translation Requests) must have the ATS Extended Capability structure in its Extended Configuration Space. It is permitted to be implemented by Endpoint Functions or RCiEPs.

ATS support is optional in SR-IOV Devices. If a VF implements the ATS Extended Capability, its associated PF must implement the ATS Extended Capability. The ATS capabilities in VFs and their associated PFs are permitted to be enabled independently.

ATS support is optional in SIOV Devices. If an SDI requires ATS functionality, its associated PF must implement the ATS Extended Capability. SDIs use the ATS settings from the ATS Extended Capability of their associated PF, however it is permitted for the SIOV Device to provide a device-specific mechanism to disable use of ATS on one or more SDIs while the ATS functionality is still enabled in the associated PF.

§ Figure 10-29 details allocation of the register fields in the ATS Extended Capability structure.

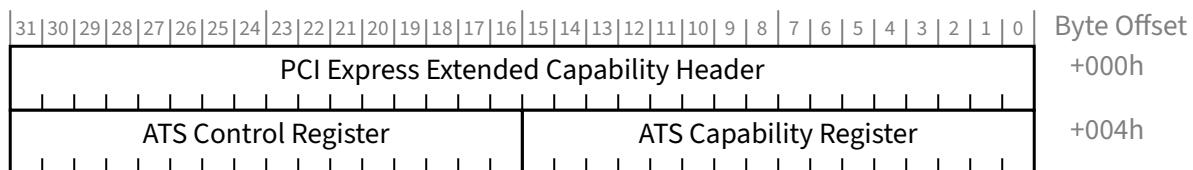


Figure 10-29 ATS Extended Capability Structure §

10.5.1.1 ATS Extended Capability Header (Offset 00h) §

§ Figure 10-30 details allocation of the register fields in the ATS Extended Capability Header; § Table 10-9 provides the respective field definitions.

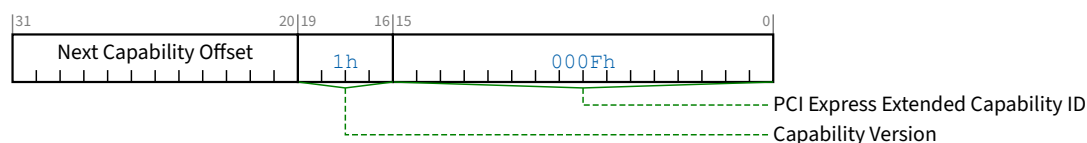


Figure 10-30 ATS Extended Capability Header §

Table 10-9 ATS Extended Capability Header §

Bit Location	Register Description	Attributes
15:0	PCI Express Extended Capability ID - Indicates the ATS Extended Capability structure. This field must return a Capability ID of "000Fh" indicating that this is an ATS Extended Capability structure.	RO
19:16	Capability Version - This field is a PCI-SIG defined version number that indicates the version of the Capability structure present. Must be "1h" for this version of the specification.	RO
31:20	Next Capability Offset - The offset to the next PCI Extended Capability structure or 000h if no other items exist in the linked list of capabilities.	RO

10.5.1.2 ATS Capability Register (Offset 04h) §

§ Figure 10-31 details the allocation of register fields of an ATS Capability Register; § Table 10-10 provides the respective bit definitions.

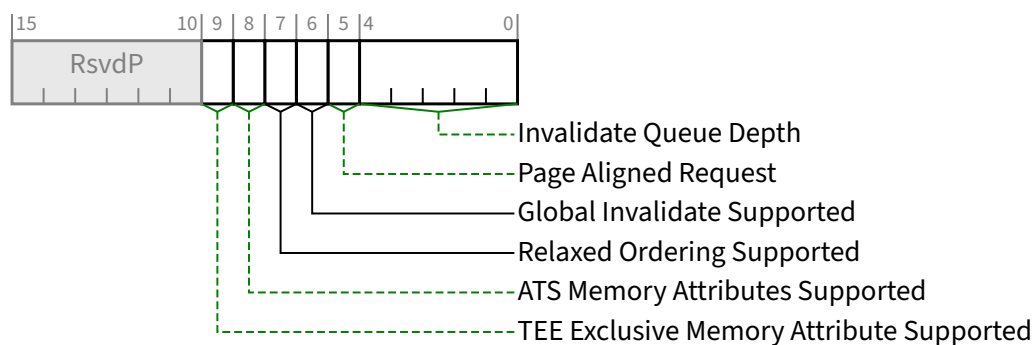


Figure 10-31 ATS Capability Register (Offset 04h) §

Table 10-10 ATS Capability Register (Offset 04h) §

Bit Location	Register Description	Attributes
4:0	Invalidate Queue Depth - The number of Invalidate Requests that the Function can accept before putting backpressure on the Upstream connection. A value of Zero in a non-VF Function indicates that it can accept 32 Invalidate Requests. For VFs, this field must be hardwired to Zero. VFs share the queue of the associated PF. See § Section 10.3.5 .	RO VF ROZ
5	Page Aligned Request - If Set, indicates the Untranslated Address is always aligned to a 4096 byte boundary. This field permits software to distinguish between implementations compatible with this specification and those compatible with an earlier version of this specification in which a Requester was permitted to supply anything in bits [11:2]. It is strongly recommended that this bit be Set.	RO
6	Global Invalidate Supported - If Set, the Function supports Invalidate Requests that have the Global Invalidate bit Set. If Clear, the Function ignores the Global Invalidate bit in all Invalidate Requests (see § Section 10.3.8). It is strongly recommended that this bit be Clear. This bit must be hardwired to Zero if the Function does not support PASID. This bit must be hardwired to Zero if the TEE-IO Supported bit is Set in the Device Capabilities Register.	RO
7	Relaxed Ordering Supported - If Set, indicates this Function is permitted to Set the RO bit in Translation Requests when Enable Relaxed Ordering bit is Set.	RO
8	ATS Memory Attributes Supported - If Set, the Function supports using AMA values from ATS Translation Completions in the Function ATC. When Clear, the Function ignores the AMA field in ATS Translation Completions.	RO
9	TEE Exclusive Memory Attribute Supported - If Set, the Function supports use of TEE Exclusive Memory Attribute from Translation Completions. If Clear, the Function ignores the TEE Exclusive Memory Attribute in all Translation Completions (see § Section 10.2.3.10). This bit must be hardwired to Zero if the Global Invalidate Supported bit is Set. This bit must be hardwired to Zero if the TEE-IO Supported bit is Clear in the Device Capabilities Register.	RO

10.5.1.3 ATS Control Register (Offset 06h) §

§ Figure 10-32 details the allocation of register fields of an ATS Control Register; § Table 10-11 provides the respective bit definitions.

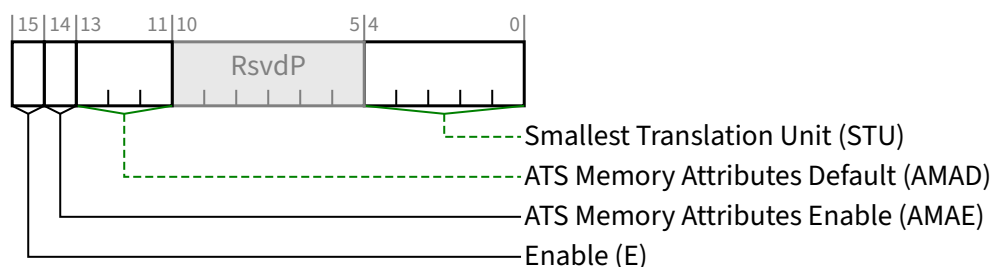


Figure 10-32 ATS Control Register §

Table 10-11 ATS Control Register §

Bit Location	Register Description	Attributes				
4:0	Smallest Translation Unit (STU) - This value indicates to the Function the minimum number of 4096-byte blocks that is indicated in a Translation Completions or Invalidate Requests. This is a power of 2 multiplier and the number of blocks is 2^{STU}. A value of 0 0000b indicates one block and a value of 1 1111b indicates 2^{31} blocks (or 8 TB total) For VFs, this field must be hardwired to Zero. The associated PF's value applies. Default value is 0 0000b.	RW VF ROZ				
13:11	ATS Memory Attributes Default (AMAD) – When Set, as a performance optimization, and when AMAE is Set, the Requester is permitted to provide only non-default AMA values in Requests using the TPH TLP Prefix. <table><tr><td>AMAD</td><td>ATS Memory Attributes</td></tr><tr><td>000b-111b</td><td>ATS Memory Attribute values.</td></tr></table> This field is permitted to be hardwired to 000b if ATS Memory Attributes Supported is Clear. Default value is 000b.	AMAD	ATS Memory Attributes	000b-111b	ATS Memory Attribute values.	RW/RO
AMAD	ATS Memory Attributes					
000b-111b	ATS Memory Attribute values.					
14	ATS Memory Attributes Enable (AMAE) - When Set, the Requester is permitted to provide AMA values in Requests using the TPH TLP Prefix. This bit is permitted to be hardwired to 0b if the ATS Memory Attributes Supported bit is Clear. Default value is 0b.	RW/RO				
15	Enable (E) - When Set, the Function is enabled to cache translations. Behavior is undefined if this Endpoint Function supports PASID, this bit is Set and any bit in the associated <u>PASID Control Register</u> is changed (see § <u>Section 7.8.9.3</u>) Default value is 0b.	RW				

10.5.2 Page Request Extended Capability Structure §

A Page Request Extended Capability Structure is used to configure the Page Request Interface mechanism. A Multi-Function Device or RCiEP Device may implement a Page Request Interface and the associated capability on any Endpoint Function within the Device. Every Page Request Interface mechanism operates independently. For SR-IOV Devices, a single Page Request Interface is permitted for the PF and is shared between the PF and its associated VFs, in

which case the PF implements this capability and its VFs must not. For SIOV Devices, a single Page Request Interface is permitted for the PF and is shared between the PF and its associated SDIs. If an SDI requires PRI functionality, its associated PF must implement the Page Request Interface and the associated capability. SDIs use the PRI settings from the Page Request Extended Capability of their associated PF, however it is permitted for the SIOV Device to provide a device-specific mechanism to disable use of PRI on one or more SDIs while the PRI functionality is still enabled in the associated PF.

For an Endpoint employing SR-IOV or SIOV, even though the Page Request Interface is shared between its PFs and VFs/SDIs, it identifies the PF, VF, or SDI via the Requester ID field of the Page Request Message, and the same ID must be in the Destination Device ID field of the resulting PRG Response Message.

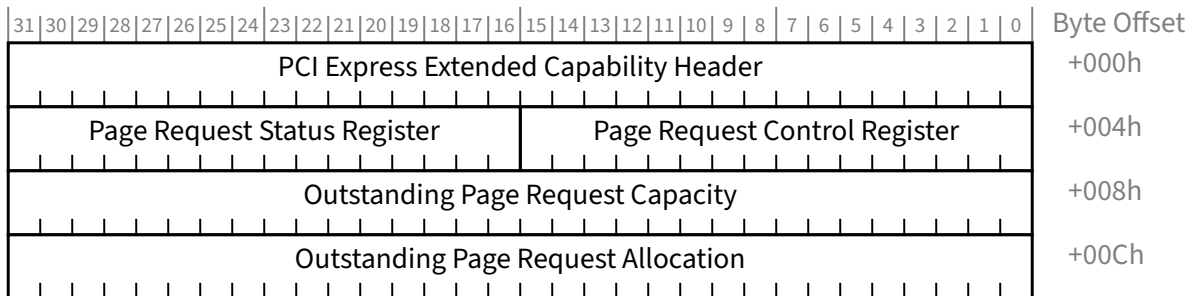


Figure 10-33 Page Request Extended Capability Structure §

10.5.2.1 Page Request Extended Capability Header (Offset 00h) §

§ Figure 10-34 details allocation of the register fields in the Page Request Extended Capability Header; § Table 10-13 provides the respective field definitions.

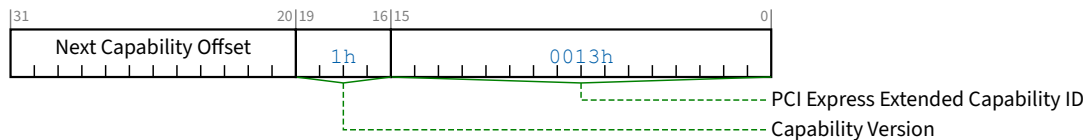


Figure 10-34 Page Request Extended Capability Header §

Table 10-13 Page Request Extended Capability Header §

Bit Location	Register Description	Attributes
15:0	PCI Express Extended Capability ID - Indicates that the associated extended capability structure is a Page Request Extended Capability. This field must return a Capability ID of "0013h".	RO
19:16	Capability Version - This field is a PCI-SIG defined version number that indicates the version of the Capability structure present. Must be "1h" for this version of the specification.	RO
31:20	Next Capability Offset - The offset to the next PCI Extended Capability structure or 000h if no other items exist in the linked list of capabilities.	RO

10.5.2.2 Page Request Control Register (Offset 04h) §

§ Figure 10-35 details allocation of the register fields in the Page Request Control Register; § Table 10-14 provides the respective field definitions.

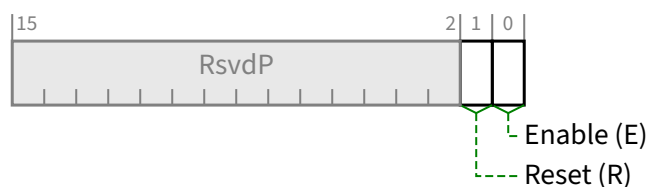


Figure 10-35 Page Request Control Register §

Table 10-14 Page Request Control Register §

Bit Location	Register Description	Attributes
0	Enable (E) - This field, when set, indicates that the Page Request Interface is allowed to make page requests. If this field is Clear, the Page Request Interface is not allowed to issue page requests. If both this field and the Stopped field are Clear, then the Page Request Interface will not issue new page requests, but has outstanding page requests that have been transmitted or are queued for transmission. When the Page Request Interface is transitioned from not-Enabled to Enabled, its status flags (Stopped, Response Failure, and Unexpected Page Request Group Index (UPRGI) flags) are cleared. Enabling a Page Request Interface that has not successfully Stopped has indeterminate results. Default value is 0b.	RW
1	Reset (R) - When the Enable field is clear, or is being cleared in the same register update that sets this field, writing a 1b to this field, clears the associated implementation dependent page request credit counter and pending request state for the associated Page Request Interface. No action is initiated if this field is written to 0b or if this field is written with any value while the Enable field is Set. Reads of this field return 0b	RW

10.5.2.3 Page Request Status Register (Offset 06h) §

§ Figure 10-36 details allocation of the register fields in the Page Request Error Register; § Table 10-15 provides the respective field definitions.

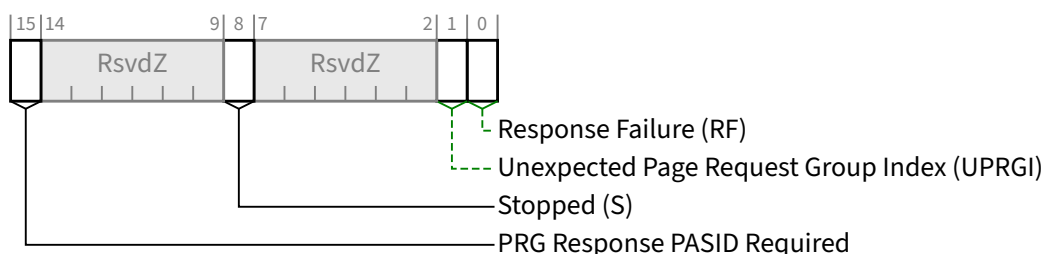


Figure 10-36 Page Request Status Register §

Table 10-15 Page Request Status Register §

Bit Location	Register Description	Attributes
0	Response Failure (RF) - This field, when Set, indicates that the Function has received a PRG Response Message indicating a Response Failure. The Function expects no further responses from the host (any received are ignored). This field is Set by the Function and Cleared when a one is written to the field. For SR-IOV or SIOV, this field is Set in the PF if the PF or any of its associated VF/SDI sharing the Page Request Interface receives a PRG Response Message indicating Response Failure. Default value is 0b.	RW1C
1	Unexpected Page Request Group Index (UPRGI) - This field, when Set, indicates that the Function has received a PRG Response Message containing a PRG index that has no matching request. This field is Set by the Function and cleared when a one is written to the field. For SR-IOV or SIOV, this field is Set in the PF if the PF or any of its associated VF/SDI sharing the Page Request Interface receives a PRG Response Message that has no matching request. Default value is 0b.	RW1C
8	Stopped (S) - When this field is Set, the associated page request interface has stopped issuing additional page requests and all previously issued Page Requests have completed. When this field is Clear the associated page request interface either has not stopped or has stopped issuing new Page Requests but has outstanding Page Requests. This field is only meaningful if Enable is Clear. If Enable is Set, this field is undefined. When the Enable field is Cleared, after having been previously Set, the interface transitions to the stopping state and Clears this field. After all page requests currently outstanding at the Function(s) have completed, this field is Set and the interface enters the disabled state. If there were no outstanding page requests, this field may be Set immediately when Enable is Cleared. Resetting the interface will cause an immediate transition to the disabled state. While in the stopping state, receipt of a Response Failure message will result in the immediate transition to the disabled state (Setting this field). For SR-IOV or SIOV, this field is Set in the PF only when the PF and all associated VFs/SDIs have stopped issuing page requests. Default value is 1b.	RO
15	PRG Response PASID Required - If Set, the Function expects a PASID on PRG Response Messages when the corresponding Page Requests had a PASID. If Clear, the Function does not expect PASID on any PRG Response Message. Function behavior is undefined if this bit is Clear and the Function receives a PRG Response Message with a PASID.	RO

Bit Location	Register Description	Attributes
	Function behavior is undefined if this bit is Set and the Function receives a PRG Response Message with no PASID when the corresponding Page Requests had a PASID. This bit is RsvdZ if the Function does not support PASID.	

10.5.2.4 Outstanding Page Request Capacity (Offset 08h) §

This register contains the number of outstanding page request messages the associated Page Request Interface physically supports. This is the upper limit on the number of pages that can be usefully allocated to the Page Request Interface.

This register is Read Only.

10.5.2.5 Outstanding Page Request Allocation (Offset 0Ch) §

This register contains the number of outstanding page request messages the associated Page Request Interface is allowed to issue (have outstanding at any given instance).

The number of PRGs a Page Request Interface has outstanding is less than or equal to the number of request messages it has issued. For example, if system software allocates 1000 messages to a Page Request Interface then a single PRG could use all 1000 of the possible requests. Conversely, at one request per PRG the Page Request Interface would run out of PRG indices (of which there are only 512) before it consumes all its page request credits. A Page Request Interface must pre-allocate its request availability for any given PRG, that is, all the requests required by a given PRG must be available before any of the requests may be issued.

When Shadow Functions are enabled, this allocation applies to the Function containing this capability and its enabled Shadow Functions (See § Section 7.9.25).

This register is Read/Write. Behavior is undefined if this register is changed while the Enable flag is set. Behavior is undefined if this register is written with a value larger than Outstanding Page Request Capacity. Default value is 0.

When PASID is supported, the Request Allocation remains associated with the Function and is shared across the Function as well as all PASIDs of the Function.

Stopping a PASID does not affect any allocation used by that PASID. The system should continue to respond with PRG Response Messages in order to return Page Request and PRG Index resources to the Function (see § Section 10.4.2.1).

Stop Marker Messages consume buffering but are not included in this allocation (see § Section 10.4.1.2.1). Systems should provide additional buffering for Stop Marker Messages and should limit the number of outstanding Stop Marker Messages to avoid overrunning this additional buffering.